



FUNDAÇÃO EDSON QUEIROZ
UNIVERSIDADE DE FORTALEZA - UNIFOR
CENTRO DE CIÊNCIAS TECNOLÓGICAS
CURSO ENGENHARIA DA COMPUTAÇÃO

**RECUPERAÇÃO E ATUALIZAÇÃO DE FRESADORA CNC COM FIRMWARE E
SOFTWARE DE CÓDIGO ABERTO PARA PRODUÇÃO DE PCBS**

GABRIEL COUTO LÓSSIO DE ALMEIDA

Fortaleza-CE

2025

GABRIEL COUTO LÓSSIO DE ALMEIDA

**RECUPERAÇÃO E ATUALIZAÇÃO DE FRESADORA CNC COM FIRMWARE E
SOFTWARE DE CÓDIGO ABERTO PARA PRODUÇÃO DE PCBS**

Trabalho de Conclusão do Curso apresentado ao curso de Engenharia da Computação da Universidade de Fortaleza, para obtenção do grau de Bacharel em Engenharia da Computação.

Orientador: Prof. Dr. Juliano de Oliveira Pacheco

Fortaleza-CE

2025

Ficha catalográfica da obra elaborada pelo autor através do programa de
geração automática da Biblioteca Central da Universidade de Fortaleza

Almeida, Gabriel Couto Lóssio de.

Recuperação e Atualização de Fresadora CNC com Firmware e
Software de Código Aberto para Produção de PCBs / Gabriel
Couto Lóssio de Almeida. - 2025
64 f.

Trabalho de Conclusão de Curso (Graduação) - Universidade
de Fortaleza. Curso de Engenharia De Computação, Fortaleza,
2025.

Orientação: Prof. Dr. Juliano de Oliveira Pacheco.

1. Retrofit. 2. CNC. 3. Fresadora de PCBs. 4. FluidNC. 5.
ESP32. I. Pacheco, Prof. Dr. Juliano de Oliveira. II. Título.



Unifor

**Graduação em
Engenharia da Computação**

TRABALHO DE CONCLUSÃO DE CURSO

RECUPERAÇÃO E ATUALIZAÇÃO DE FRESADORA CNC COM *FIRMWARE* E SOFTWARE DE CÓDIGO ABERTO PARA PRODUÇÃO DE PCBs

Monografia apresentada à Universidade de Fortaleza como requisito de avaliação para
obtenção do título de Graduação em Engenharia da Computação

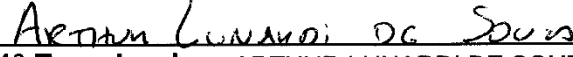

GABRIEL COUTO LÓSSIO DE ALMEIDA

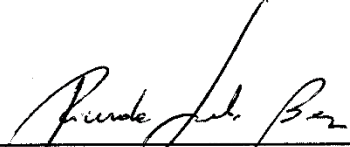
Monografia defendida em 04/12/2025

Resultado:

Banca examinadora


Orientador: JULIANO DE OLIVEIRA PACHECO, Dr.


1º Examinador: ARTHUR LUNARDI DE SOUZA, Me.


2º Examinador: Eng.º RICARDO SILVA BEZERRA



Prof. Me. André Lunardi de Souza
Coordenador do Curso de Engenharia da Computação

AGRADECIMENTOS

Agradeço primeiramente a Deus, por me conceder saúde, força e perseverança para superar os desafios ao longo deste curso e, em especial, durante o desenvolvimento deste trabalho.

À minha família, pelo apoio incondicional, incentivo constante e por nunca medirem esforços para que eu pudesse alcançar meus objetivos, mesmo diante das dificuldades.

À minha namorada, pelo carinho, paciência, compreensão e apoio emocional durante toda essa jornada, sendo fundamental nos momentos mais desafiadores.

Ao meu orientador, Prof. Dr. Juliano de Oliveira Pacheco, pela orientação técnica, paciência, dedicação e valiosas contribuições ao longo de todas as etapas deste trabalho, sendo fundamental para a sua conclusão.

Ao Engenheiro Ricardo Bezerra, pela valiosa colaboração ao ceder a placa controladora MKS e os drivers TB6600, recursos essenciais para a realização prática do projeto.

Aos professores do curso de Engenharia de Computação da Universidade de Fortaleza (Unifor), pelos ensinamentos transmitidos durante a graduação, que foram essenciais para minha formação acadêmica e profissional.

Aos amigos e colegas que, de forma direta ou indireta, contribuíram com apoio, sugestões, testes e incentivo ao longo do desenvolvimento deste projeto.

Por fim, a todos que de alguma forma participaram dessa etapa da minha vida, minha sincera gratidão.

“Tudo parece impossível até que seja feito. ”

Nelson Mandela

RESUMO

Este trabalho apresenta o processo de retrofit de uma fresadora de placas de circuito impresso (PCBs) LPKF ProtoMat C60 que se encontrava inoperante no laboratório L4 da Universidade de Fortaleza devido à falha irreversível de seu sistema de controle proprietário. Apesar de sua estrutura mecânica estar preservada, a ausência de um controlador funcional impossibilitava o uso do equipamento, motivando a substituição completa da eletrônica por uma solução de baixo custo baseada no microcontrolador ESP32 e no firmware de código aberto FluidNC. O estudo abrange o diagnóstico inicial da máquina, a seleção dos componentes, a integração eletrônica e a configuração do novo sistema de controle, incluindo drivers TB6600 para os eixos X e Y, um módulo MOSFET para o solenoide do eixo Z e o desenvolvimento de um Controlador Eletrônico de Velocidade para o spindle, implementado em um Arduino Uno por meio de técnicas de síntese digital direta. Após a montagem, foram realizados testes de homing, movimentação por G-code, calibração dos eixos e acionamento do spindle, todos executados com sucesso. A validação prática ocorreu por meio de uma impressão preliminar com caneta e, posteriormente, pela usinagem completa de uma placa de circuito impresso, confirmando a precisão dimensional e o funcionamento estável da máquina retrofitada. Os resultados demonstram que a modernização devolveu utilidade ao equipamento, oferecendo maior flexibilidade, facilidade de configuração e compatibilidade com fluxos modernos de fabricação. O trabalho conclui que o retrofit constitui uma alternativa eficaz e economicamente vantajosa em relação à substituição da máquina, além de contribuir para o aprendizado em automação, sistemas embarcados e CNC. São também apresentadas sugestões de melhorias, como a implementação de um sistema de aspiração, o enclausuramento da eletrônica e a substituição futura do ESC por um modelo com realimentação, visando aumentar a confiabilidade e a durabilidade do equipamento.

Palavras-chave: Retrofit; CNC; ESP32; FluidNC; Fresadora de PCBs.

ABSTRACT

This work presents the retrofit of an LPKF ProtoMat C60 printed circuit board (PCB) milling machine that had become inoperative in the L4 laboratory of the University of Fortaleza due to the irreversible failure of its proprietary control system. Although the mechanical structure remained intact, the absence of a functional controller prevented the equipment from being used, motivating the complete replacement of its electronics with a low-cost solution based on an ESP32 microcontroller and the open-source firmware FluidNC. The study includes the initial diagnostic assessment of the machine, component selection, electronic integration, and configuration of the new control system, which incorporates TB6600 drivers for the X and Y axes, a MOSFET module for the Z-axis solenoid, and the development of a custom Electronic Speed Controller for the spindle, implemented on an Arduino Uno using Direct Digital Synthesis techniques. After assembly, homing tests, G-code motion commands, axis calibration, and spindle control were successfully performed. Practical validation was achieved through an initial pen-plotting test followed by the complete milling of a PCB, confirming dimensional accuracy and stable operation of the retrofitted machine. The results demonstrate that the modernization restored full functionality to the equipment while providing greater flexibility, easier configuration, and compatibility with modern manufacturing workflows. The study concludes that the retrofit is an effective and economically advantageous alternative to machine replacement, contributing to hands-on learning in automation, embedded systems, and CNC technologies. The work also proposes future improvements, including the addition of a dust extraction system, protective housing for the electronics, and the replacement of the current ESC with a feedback-enabled model to enhance reliability and durability.

Keywords: Retrofit; CNC; ESP32; FluidNC; PCB milling.

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Justificativa	13
1.2	Objetivos.....	13
1.3	Estrutura do Trabalho	14
2	REFERENCIAL TEÓRICO.....	15
2.1	Retrofit de Máquinas CNC	15
2.2	Funcionamento de Fresadoras de PCBs	15
2.3	ESP32 como Plataforma de Controle CNC	15
2.4	Fresadora LPKF Protomat C60	16
2.5	Firmwares de Código Aberto para CNC.....	17
2.6	Linguagem G-code e Controle de Máquinas CNC.....	18
2.7	Motores BLDC	19
2.8	Componentes Adicionados	22
3	METODOLOGIA.....	25
3.1	Levantamento de Informações e Diagnóstico.....	25
3.2	Pesquisa Bibliográfica e Técnica.....	25
3.3	Seleção e Configuração do Sistema de Controle.....	25
3.4	Montagem e Integração do Sistema.....	25
3.5	Calibração e Testes de Operação	26
3.6	Documentação e Análise dos Resultados	26
4	DESENVOLVIMENTO	27
4.1	Situação Inicial	27
4.2	Escolha do Firmware	30
4.3	Planejamento do Retrofit	31
4.4	Montagem e Integração do Sistema.....	33

4.5	FluidNC	35
4.6	Controlador Eletrônico de Velocidade Desenvolvido.....	42
5	RESULTADOS E DISCUSSÕES	45
5.1	Homing	45
5.2	Testes Funcionais dos Eixos e do Spindle via G-code	46
5.3	Calibração dos Eixos da Fresadora.....	47
5.4	Geração dos arquivos G-code a partir do KiCad e FlatCAM.....	48
5.5	Validação Preliminar do Processo por Impressão com Caneta.....	49
5.6	Usinagem Final da Placa de Circuito Impresso	50
5.7	Propostas de Melhorias para a Fresadora Retrofitada	52
6	CONCLUSÃO.....	54

1 INTRODUÇÃO

A fabricação de placas de circuito impresso (PCBs) é uma etapa essencial no desenvolvimento de dispositivos eletrônicos, permitindo a prototipagem e a validação de circuitos antes da produção em larga escala. Em laboratórios acadêmicos, fresadoras de PCBs desempenham um papel fundamental ao possibilitar a fabricação rápida de protótipos, reduzindo a dependência de fornecedores externos e acelerando o ciclo de desenvolvimento de projetos. No entanto, a obsolescência de componentes eletrônicos e falhas nos sistemas de controle podem tornar esses equipamentos inoperantes, comprometendo sua utilidade e demandando soluções de modernização.

O presente trabalho propõe o retrofit de uma fresadora de PCBs localizada no laboratório L4 da Unifor, que se encontra inoperante devido à falha do seu sistema de controle original. Embora a estrutura mecânica da máquina, incluindo a fresa e os motores de passo, ainda esteja funcional, a ausência de um controlador impede seu uso. Para solucionar esse problema, será implementado um novo sistema de controle baseado em um microcontrolador ESP32. Essa abordagem visa restaurar e até aprimorar as funcionalidades da fresadora, tornando-a compatível com fluxos de trabalho modernos de usinagem CNC.

A relevância deste projeto está na recuperação de um equipamento essencial para atividades acadêmicas e experimentais, promovendo o aprendizado prático sobre automação, fabricação de PCBs e integração de sistemas de controle. Além disso, o estudo contribui para o conhecimento sobre retrofit de máquinas CNC, destacando o uso de soluções de baixo custo e software de código aberto para revitalizar equipamentos obsoletos.

Com base nesse contexto, este trabalho tem como objetivo principal restaurar o funcionamento da fresadora por meio da substituição do sistema de controle original, garantindo sua operação confiável e eficiente. Para isso, serão exploradas técnicas de automação, integração de hardware e software e ajustes de parâmetros de usinagem, visando otimizar a precisão e o desempenho da máquina.

1.1 Justificativa

A fresadora de PCBs do laboratório L4 da Unifor encontra-se inoperante devido à falha do sistema de controle original, embora seus componentes mecânicos, como spindle e motores de passo, estejam em bom estado. Substituir toda a máquina implicaria em custos elevados, enquanto o retrofit representa uma alternativa viável, econômica e sustentável.

A proposta de utilizar um ESP32 com um firmware de código aberto permite modernizar o equipamento com uma solução de baixo custo e fácil integração, restaurando sua funcionalidade e adaptando-a a fluxos modernos de fabricação. Essa iniciativa beneficia diretamente as atividades acadêmicas e práticas dos cursos de Engenharia, além de promover o aprendizado em automação, CNC e sistemas embarcados.

Dessa forma, o trabalho se justifica pela recuperação de um recurso didático importante, pela economia de recursos institucionais e pelo valor educacional agregado à formação dos alunos.

1.2 Objetivos

Objetivo Geral:

Desenvolver e implementar o retrofit de uma fresadora de placas de circuito impresso (PCBs) inoperante no laboratório L4 da Unifor, substituindo o sistema de controle original, de modo a restabelecer seu funcionamento, modernizar a interface de operação e possibilitar a produção precisa de protótipos de PCBs.

Objetivos Específicos:

- Diagnosticar o estado atual da fresadora de PCBs e identificar os componentes aproveitáveis para o retrofit.
- Comparar diferentes firmwares de código aberto para escolher o que melhor controla a fresadora.
- Implementar a comunicação entre o ESP32, os drivers dos motores de passo, o controle do spindle e do solenoide da máquina.
- Implementar técnicas de controle de velocidade para o motor sem escovas usado na fresa.

- Realizar testes de calibração, movimentação e fresagem para validação da funcionalidade da máquina.
- Documentar o processo de retrofit, incluindo esquemas, configurações, códigos e ajustes necessários.

1.3 Estrutura do Trabalho

Este trabalho caracteriza-se como uma pesquisa experimental, com apoio bibliográfico e documental. A pesquisa bibliográfica será utilizada para embasar teoricamente os conceitos de retrofit, controle numérico computadorizado (CNC), sistemas embarcados e utilização de firmwares de código aberto para controle de máquinas. A pesquisa documental será aplicada na análise de manuais técnicos, esquemas elétricos e registros do equipamento existente.

Como instrumento de coleta de dados, será realizada a análise prática e técnica do equipamento, com levantamento das condições mecânicas e eletrônicas da fresadora. Também serão utilizadas observações diretas durante o processo de montagem e testes do sistema de controle.

As técnicas de análise de dados incluem a avaliação funcional do equipamento após o retrofit, com base em testes de movimentação, usinagem e precisão. A análise será feita por comparação entre os parâmetros esperados e os resultados obtidos, considerando a confiabilidade, repetibilidade e qualidade dos protótipos produzidos.

2 REFERENCIAL TEÓRICO

2.1 Retrofit de Máquinas CNC

O termo retrofit refere-se ao processo de modernização de máquinas ou sistemas antigos por meio da substituição de componentes obsoletos por tecnologias mais atuais. No contexto da manufatura, especialmente em máquinas CNC, o retrofit visa restaurar a funcionalidade e melhorar o desempenho de equipamentos que, embora estruturalmente operacionais, sofreram falhas em seus sistemas de controle ou se tornaram incompatíveis com tecnologias atuais.

Em laboratórios acadêmicos e ambientes industriais, o retrofit é uma alternativa viável à aquisição de novos equipamentos, principalmente quando os custos são limitados. Além de restaurar a operação da máquina, esse processo possibilita a integração com softwares modernos, controle mais preciso e maior segurança operacional. O retrofit de fresadoras de PCBs permite, por exemplo, que equipamentos antigos voltem a ser utilizados em projetos de prototipagem rápida de circuitos eletrônicos.

2.2 Funcionamento de Fresadoras de PCBs

As fresadoras de PCBs são máquinas CNC projetadas para remover material de placas revestidas de cobre, esculpindo trilhas e ilhas que formam o circuito. Esse tipo de equipamento normalmente utiliza motores de passo para movimentar a ponte e a mesa em três eixos (X, Y, Z), além de um spindle (motor de alta rotação) para a usinagem do cobre. O controle desses movimentos é realizado por meio de comandos G-code enviados por um computador ou microcontrolador.

A precisão da fresadora é essencial para garantir a integridade elétrica dos circuitos e a fidelidade ao projeto eletrônico. Por isso, a escolha de componentes de controle confiáveis e de um firmware adequado é crucial para o sucesso do retrofit.

2.3 ESP32 como Plataforma de Controle CNC

O ESP32 é um microcontrolador de alto desempenho, com arquitetura dual-core, Wi-Fi e Bluetooth integrados, e diversos periféricos digitais. Sua ampla capacidade de processamento e conectividade torna-o uma alternativa acessível e moderna para projetos de automação e controle CNC.

Na aplicação em fresadoras, o ESP32 pode assumir o papel de unidade central de controle, processando comandos G-code e convertendo-os em sinais para drivers de motores de passo e controle do spindle. Sua versatilidade permite integração com interfaces web, cartões SD, displays e softwares de envio de G-code.

2.4 Fresadora LPKF Protomat C60

A LPKF ProtoMat C60 é uma fresadora de precisão projetada para a fabricação de placas de circuito impresso (PCBs) em pequena escala. O equipamento utiliza um sistema de movimentação baseado em motores de passo nos eixos X e Y, que garantem posicionamento preciso da ferramenta e repetibilidade nas trajetórias de usinagem. O eixo Z da máquina não utiliza motor de passo, mas sim um solenoide eletromecânico, responsável por acionar verticalmente a ferramenta de corte de forma rápida e controlada, permitindo o contato ou afastamento do spindle em relação à superfície da placa.

O spindle é um motor de alta rotação, geralmente operando entre 20.000 e 60.000 rpm, responsável pela remoção do material condutor através de microfresas. Sua elevada velocidade é essencial para manter a qualidade das trilhas e reduzir o desgaste das ferramentas. A combinação entre os motores de passo, o solenoide e o spindle confere à ProtoMat C60 alta precisão e eficiência na prototipagem de circuitos, tornando-a uma referência em equipamentos de usinagem de PCBs para fins educacionais e laboratoriais.

Seus principais componentes são:

- Motor de Passo

A Figura 1 apresenta o motor de passo utilizado na fresadora. Trata-se de um tipo de motor elétrico usado em máquinas CNC que converte pulsos elétricos em movimentos mecânicos precisos e controlados. Cada pulso faz o eixo do motor girar um ângulo fixo, permitindo movimentações exatas e repetitivas nos eixos da máquina (X, Y). Isso garante alta precisão no posicionamento das ferramentas ou do material, essencial para cortes, gravações e usinagens detalhadas.

- Solenoide

A Figura 2 apresenta o solenoide utilizado na fresadora. Trata-se de um atuador eletromecânico que converte energia elétrica em movimento linear. Ele é usado para mover a fresa para cima e para baixo no eixo Z. Quando recebe corrente elétrica, o campo magnético interno puxa um

êmbolo metálico, fazendo a fresa descer; ao desligar a corrente, uma mola a retorna à posição inicial.

- Spindle

A Figura 3 apresenta o spindle utilizado na fresadora. Ele é o motor principal da fresadora, responsável por girar a fresa em alta velocidade para cortar, perfurar ou usinar o material. Ele determina a potência e a precisão do corte, sendo o componente que efetivamente realiza o trabalho de usinagem.

- Interruptor de Fim de Curso

São sensores que detectam quando o eixo da CNC chega ao limite de movimento. Eles servem para evitar colisões e danos, enviando um sinal ao sistema para parar o motor assim que o eixo atinge o ponto máximo permitido.



Figura 1-Motor de Passo



Figura 2 - Solenoide



Figura 3 - Spindle

2.5 Firmwares de Código Aberto para CNC

Nos últimos anos, diversos firmwares de código aberto foram desenvolvidos para o controle de máquinas CNC. Entre eles, destacam-se o Grbl, FluidNC, Marlin (em adaptações CNC) e Klipper (mais voltado a impressoras 3D, mas com aplicações CNC). Esses firmwares oferecem suporte a G-code, configuração de parâmetros por arquivos ou interface, e compatibilidade com uma grande variedade de controladores e drivers.

A adoção de um firmware de código aberto no retrofit da fresadora proporciona flexibilidade, acesso ao código-fonte, facilidade de manutenção e adaptação às necessidades específicas do projeto. Além disso, permite a integração com softwares gratuitos e amplamente utilizados na comunidade maker e acadêmica.

O FluidNC, por exemplo, é um firmware leve, modular e otimizado para rodar em ESP32, oferecendo suporte a diversas combinações de hardware, com comunicação via Wi-Fi e interface web para configuração e controle. Sua estrutura modular e facilidade de personalização tornam-no uma opção atrativa para retrofits.

2.6 Linguagem G-code e Controle de Máquinas CNC

O G-code é a linguagem padrão utilizada para o controle de máquinas CNC. Por meio dela, são descritos os movimentos, trajetórias e parâmetros de operação que a máquina deve executar, funcionando como um elo entre o software de usinagem (CAM) e o firmware controlador.

Cada instrução do G-code é composta por letras seguidas de valores numéricos, representando comandos específicos. As principais são:

- G: define o tipo de movimento (ex.: G00 – movimento rápido, G01 – movimento linear, G02/G03 – movimentos circulares);
- M: executa funções auxiliares (ex.: M03 – liga o spindle, M05 – desliga);
- X, Y, Z: coordenadas dos eixos;
- F: taxa de avanço;
- S: rotação do spindle;
- T: seleção de ferramenta.

Esses comandos são processados sequencialmente pelo firmware, que os converte em sinais elétricos para os motores e atuadores. O sistema pode operar em modo absoluto (G90), referenciando-se à origem da máquina, ou incremental (G91), com base na posição atual.

O firmware FluidNC, por exemplo, interpreta diretamente o G-code e controla os motores de passo, spindle e periféricos da máquina em tempo real. Ele gerencia a cinemática, acelerações e limites de segurança, garantindo que os movimentos sigam com precisão o que foi programado.

O uso do G-code permite automatizar e padronizar o controle de máquinas CNC, tornando possível reproduzir percursos complexos e ajustar parâmetros de operação com alta precisão e repetibilidade.

2.7 Motores BLDC

Princípios de Funcionamento

Os motores BLDC (Brushless DC Motor), ou motores de corrente contínua sem escovas, são dispositivos amplamente utilizados em aplicações que exigem alta eficiência, baixo ruído e controle preciso de velocidade e torque. Diferentemente dos motores DC convencionais, o BLDC dispensa o uso de escovas e comutadores mecânicos, substituídos por um sistema eletrônico de comutação, o que reduz o desgaste e aumenta a vida útil do equipamento.

A estrutura básica de um motor BLDC é composta por um rotor com ímãs permanentes e um estator trifásico com enrolamentos distribuídos uniformemente. Quando as bobinas do estator são energizadas em sequência, forma-se um campo magnético girante, que interage com o campo magnético fixo do rotor, produzindo o torque necessário para a rotação.

A comutação eletrônica das fases é controlada por um circuito denominado ESC (Electronic Speed Controller). Esse controlador determina a sequência e o tempo de energização das fases, substituindo o comutador mecânico presente nos motores DC tradicionais. O ESC pode operar em dois modos principais:

- Controle com realimentação (closed-loop): Utiliza sensores de posição (como sensores Hall) ou medições da força contraeletromotriz (back-EMF) para sincronizar com precisão o campo do estator à posição do rotor, proporcionando alto torque e eficiência.
- Controle sem realimentação (open-loop): Realiza a comutação das fases em uma sequência pré-definida, sem utilizar sensores. Nesse modo, o sistema não monitora a posição do rotor, mas ainda é capaz de gerar o campo magnético girante necessário para o funcionamento. Normalmente, inicia-se com uma frequência de comutação baixa, que é gradualmente aumentada até a velocidade desejada, permitindo que o rotor se alinhe de forma suave com o campo magnético.

O controle open-loop é mais simples e econômico, porém apresenta limitações em baixas rotações e na resposta a variações de carga, já que não há correção automática de sincronismo. Ainda assim, esse tipo de acionamento é amplamente utilizado em sistemas de menor complexidade, testes experimentais e aplicações onde o controle preciso de torque não é essencial.

Controle Eletrônico

O funcionamento dos motores BLDC depende diretamente de um sistema de controle eletrônico responsável por comutar as fases do estator de forma sincronizada, mantendo a rotação do campo magnético em relação ao rotor. Esse sistema é comumente denominado ESC (Electronic Speed Controller), ou controlador eletrônico de velocidade. O ESC tem como principais funções:

- Comandar a sequência de energização das fases do motor de acordo com a posição do rotor;
- Ajustar a velocidade de rotação por meio da modulação da tensão ou da frequência aplicada;
- Controlar o torque fornecido, regulando a corrente nas bobinas.

Etapas de Controle

O controle eletrônico de um motor BLDC envolve três etapas fundamentais:

- Comutação das Fases – O ESC ativa os enrolamentos do estator em uma sequência específica, criando um campo magnético girante. Essa sequência normalmente segue um padrão trifásico, em que apenas duas fases são energizadas simultaneamente enquanto a terceira permanece desligada, sendo alternadas ciclicamente.
- Modulação de Velocidade (PWM) – A velocidade de rotação do motor é controlada variando-se a largura dos pulsos de tensão aplicados às bobinas, por meio da técnica de PWM (Pulse Width Modulation). Alterando o ciclo de trabalho do PWM, é possível ajustar a tensão média e, consequentemente, a velocidade de rotação.
- Sincronização com o Rotor – Em sistemas com realimentação (closed-loop), sensores de efeito Hall ou medições de back-EMF informam ao controlador a posição exata do rotor, permitindo que a comutação ocorra no momento ideal. Em contrapartida, no controle sem realimentação (open-loop), a comutação é realizada segundo um padrão

de tempo pré-definido, e a velocidade é controlada exclusivamente pela frequência de comutação.

Circuito de Potência

Para realizar a comutação das fases, o ESC utiliza um conjunto de transistores MOSFET dispostos em uma configuração denominada ponte trifásica. Cada par de MOSFETs controla o fluxo de corrente em uma das três fases do motor, conforme mostrado na Figura 4. Esses transistores atuam como chaves eletrônicas, permitindo ou bloqueando a passagem de corrente conforme os sinais de controle gerados pelo microcontrolador.

O uso de MOSFETs é vantajoso devido à sua alta velocidade de chaveamento, baixa resistência interna ($R_{ds(on)}$) e eficiência energética, características que minimizam perdas de potência e aquecimento durante o funcionamento do motor.

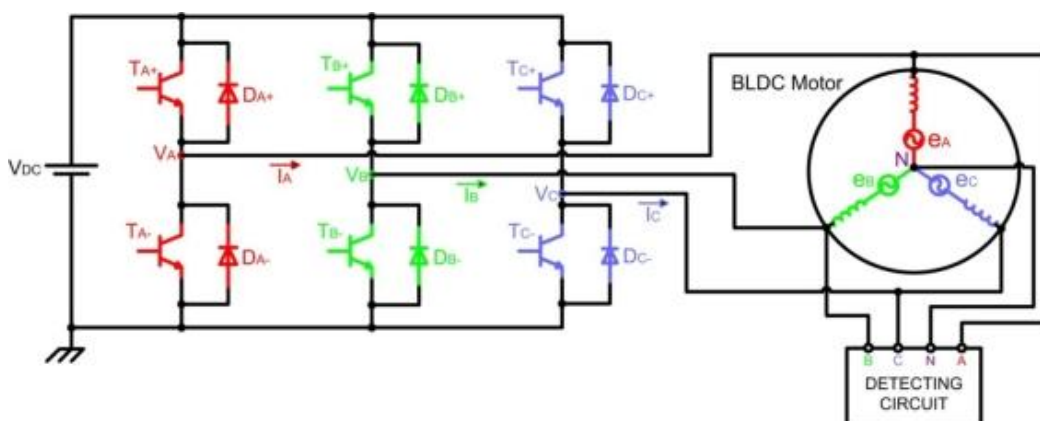


Figura 4 - Estrutura típica de driver para motor BLDC

Fonte: GAMAZO-REAL; VÁZQUEZ-SÁNCHEZ; GÓMEZ-GIL (2010)

O controle eletrônico é o elemento que diferencia o motor BLDC de outros tipos de máquinas elétricas. Ele substitui o sistema de comutação mecânica tradicional por uma comutação digital precisa, ajustável via software e capaz de integrar-se a sistemas embarcados.

Mesmo em configurações open-loop, o ESC permite o acionamento e o controle de velocidade de motores BLDC com eficiência, sendo uma solução versátil para aplicações que não demandam sensoramento de posição.

2.8 Componentes Adicionados

- Placa Principal de Controle CNC: MKS DLC32

A Figura 5 apresenta a placa controladora MKS DLC32 utilizada neste projeto. A placa MKS DLC32 é uma placa controladora desenvolvida para sistemas CNC e máquinas de gravação a laser, oferecendo uma solução moderna, compacta e de fácil integração. Baseada em um microcontrolador ESP32, ela combina alto desempenho de processamento com conectividade Wi-Fi e Bluetooth, permitindo o controle remoto da máquina por meio de interfaces gráficas ou navegadores web. A MKS DLC32 é compatível com firmwares como o FluidNC, que possibilitam o uso de comandos G-code para controle preciso dos eixos e periféricos. Além disso, a placa conta com saídas dedicadas para drivers de motores de passo, controle de spindle ou laser, fim de curso, botões de parada de emergência, e outras interfaces auxiliares. Sua arquitetura aberta e suporte à comunidade tornam-na ideal para projetos de retrofit e personalização de máquinas CNC, substituindo controladores proprietários por uma plataforma flexível, atual e de fácil configuração.

- Driver para Motor de Passo: TB6600

A Figura 6 apresenta o driver TB6600 usado neste projeto. Trata-se de um driver amplamente utilizado para o controle de motores de passo bipolares, oferecendo alta confiabilidade e precisão no acionamento dos eixos de máquinas CNC, impressoras 3D e sistemas automatizados. Ele é capaz de operar com correntes de até 4 A por fase e tensões entre 9 e 42 V, tornando-se compatível com uma ampla variedade de motores de passo. O TB6600 recebe sinais de Step e Direction provenientes da placa controladora (como a MKS DLC32) e converte esses sinais em pulsos de corrente controlados para as bobinas do motor, garantindo movimentos suaves e precisos. Além disso, o driver permite configurar parâmetros como microstepping e corrente de saída, por meio de chaves DIP, o que possibilita ajustar o desempenho de acordo com as necessidades do projeto. Sua robustez, simplicidade de uso e boa dissipação térmica fazem do TB6600 uma escolha ideal para projetos de retrofit de fresadoras, garantindo o controle estável e seguro dos eixos da máquina.

- Módulo MOSFET Optoacoplado FR120N

A Figura 8 apresenta o módulo MOSFET optoacoplado FR120N (100 V / 9,4 A), utilizado para realizar o acionamento do solenoide da fresadora de forma segura e isolada eletricamente do circuito de controle. Esse módulo utiliza um optoacoplador para separar o sinal lógico, proveniente do microcontrolador (como o ESP32 ou Arduino), do circuito de potência responsável por conduzir a corrente do solenoide. Dessa forma, evita-se que ruídos elétricos ou picos de corrente retornem ao sistema de controle, garantindo maior segurança e confiabilidade. O MOSFET FR120N, presente no módulo, possui alta capacidade de corrente e baixa resistência interna, permitindo o acionamento eficiente de cargas indutivas, como solenoides e motores DC. Essa configuração possibilita um comando rápido e estável, com menor dissipação de calor, sendo ideal para aplicações em retrofit de máquinas CNC, onde há necessidade de acionar dispositivos de potência a partir de sinais de baixa tensão.

- Controle do Spindle: Arduino Uno

A Figura 7 apresenta o Arduino Uno utilizado neste projeto. Trata-se de uma placa de prototipagem baseada no microcontrolador ATmega328P, que opera a 16 MHz e possui 32 KB de memória flash, 2 KB de SRAM e 1 KB de EEPROM. Conta com 14 pinos digitais, 6 entradas analógicas e interface USB para programação e comunicação.

Seu microcontrolador dispõe de temporizadores internos com contadores e comparadores que permitem medir intervalos de tempo, gerar sinais PWM e controlar eventos com precisão. São três timers (dois de 8 bits e um de 16 bits) com registradores de comparação (OCR), usados para comparar valores e acionar interrupções ou saídas automáticas.

Essas funções tornam o Arduino Uno ideal para aplicações de controle, automação e geração de sinais, como em sistemas de motores e máquinas CNC.

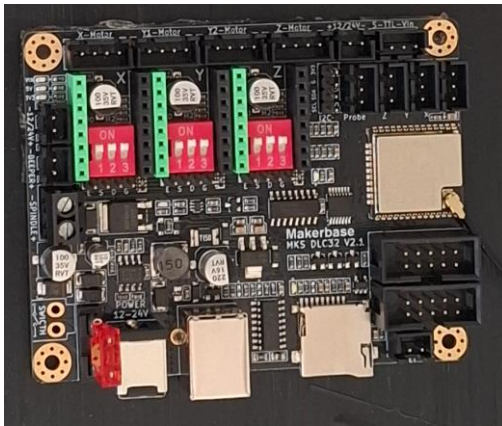


Figura 5 - Placa MKS DLC32



Figura 6 - Driver TB6600

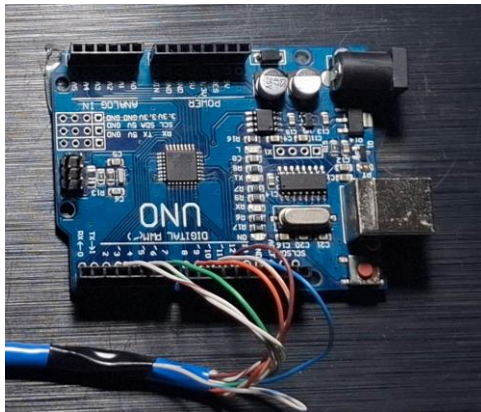


Figura 7 – Placa Arduino Uno

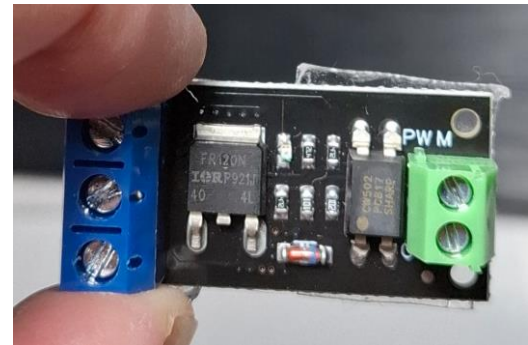


Figura 8 - Módulo MOSFET
Optoacoplado FR120N

3 METODOLOGIA

Este trabalho adota uma abordagem prática e experimental, dividida em etapas bem definidas que visam restaurar o funcionamento de uma fresadora de PCBs por meio de um processo de retrofit utilizando um microcontrolador ESP32 com firmware de código aberto.

A metodologia será estruturada nas seguintes fases:

3.1 Levantamento de Informações e Diagnóstico

Nesta etapa inicial, será feito o levantamento do estado atual da fresadora, avaliando os componentes mecânicos e eletrônicos ainda funcionais, como motores de passo, spindle, fonte de alimentação, drivers e estrutura da máquina. Também será analisado o histórico do equipamento e sua documentação técnica, se disponível.

3.2 Pesquisa Bibliográfica e Técnica

Será realizada uma pesquisa bibliográfica e documental para embasar teoricamente o projeto, abordando temas como retrofit de máquinas CNC, firmwares de código aberto para controle com ESP32, controle de motores de passo, comunicação via G-code, e uso de ferramentas de usinagem para PCBs.

3.3 Seleção e Configuração do Sistema de Controle

Com base na pesquisa, será escolhido o firmware de código aberto mais adequado para o ESP32. Serão configurados o firmware, a pinagem e os parâmetros de controle dos eixos, além da integração com drivers dos motores de passo e controle do spindle.

3.4 Montagem e Integração do Sistema

A nova eletrônica de controle será instalada, e os testes iniciais de movimentação e resposta dos eixos serão realizados. A fresadora será conectada a um computador para receber comandos G-code por meio de softwares compatíveis com o firmware escolhido.

3.5 Calibração e Testes de Operação

Serão realizados testes práticos para calibração dos eixos, validação do movimento e testes de fresagem de PCBs com diferentes configurações. Os resultados serão registrados para análise de precisão, confiabilidade e desempenho da máquina.

3.6 Documentação e Análise dos Resultados

Todo o processo de retrofit será documentado, incluindo esquemas de ligação, configurações do firmware, testes realizados e ajustes aplicados. Os resultados obtidos serão comparados aos objetivos propostos, com análise qualitativa e quantitativa do desempenho da máquina após o retrofit.

4 DESENVOLVIMENTO

4.1 Situação Inicial

A Figura 9 apresenta a fresadora de placas de circuito impresso em seu estado inicial, conforme se encontrava no laboratório L04 da Unifor. O equipamento estava em estado de abandono, completamente desmontado, com suas placas eletrônicas e componentes dispostos sobre a bancada, fora da estrutura principal da máquina. Observou-se que, ao longo do tempo, diferentes tentativas de reparo foram realizadas por outros estudantes e técnicos, porém sem sucesso em restabelecer seu funcionamento.

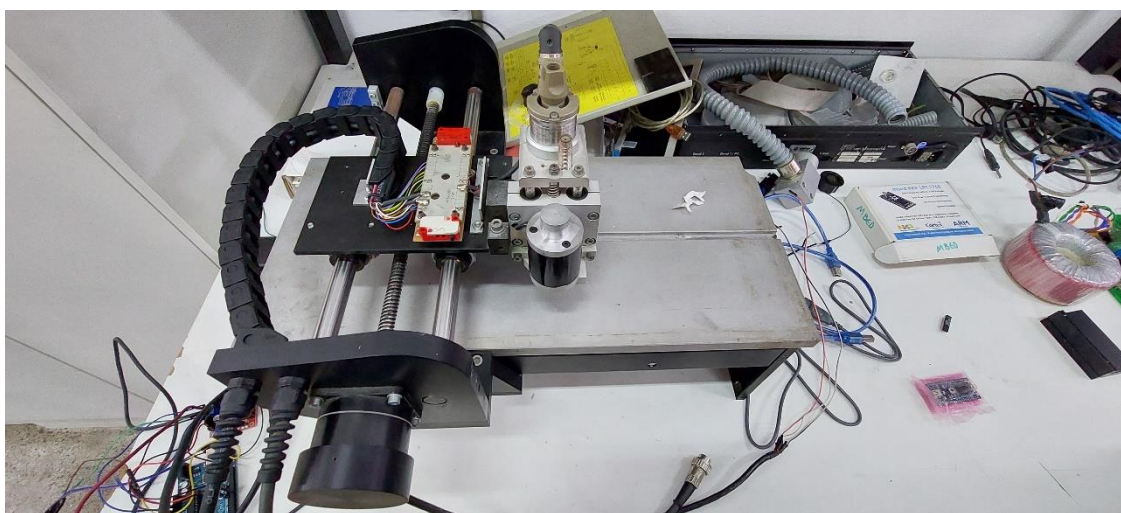


Figura 9 - Situação Inicial da Fresadora

Essa condição inicial dificultava a identificação precisa das conexões originais e do estado real de cada componente, tornando necessário um processo detalhado de diagnóstico e reestruturação do sistema eletrônico. Diante desse cenário, o projeto de retrofit foi proposto como alternativa viável para recuperar e modernizar o equipamento, restabelecendo sua funcionalidade e integrando novos recursos de controle.

➤ Levantamento de Informações e Diagnósticos dos Componentes da Máquina:

4.1.1 Placa Principal

A Figura 10 apresenta a placa principal da fresadora, responsável por atuar como o centro de controle de todo o sistema, sendo responsável por coordenar a comunicação entre o computador

e os componentes eletromecânicos da máquina. É ela quem interpreta os comandos enviados pelo software de usinagem e converte essas instruções em sinais elétricos que controlam o movimento dos motores de passo nos eixos X, Y além de acionar dispositivos auxiliares, como o spindle e o solenoide. Essa placa também realiza a distribuição das tensões de alimentação necessárias para o funcionamento dos módulos conectados, garantindo que cada parte da fresadora opere de forma sincronizada e segura. Em suma, trata-se do elemento essencial que integra o controle lógico, o acionamento dos atuadores e a comunicação com o sistema computacional.

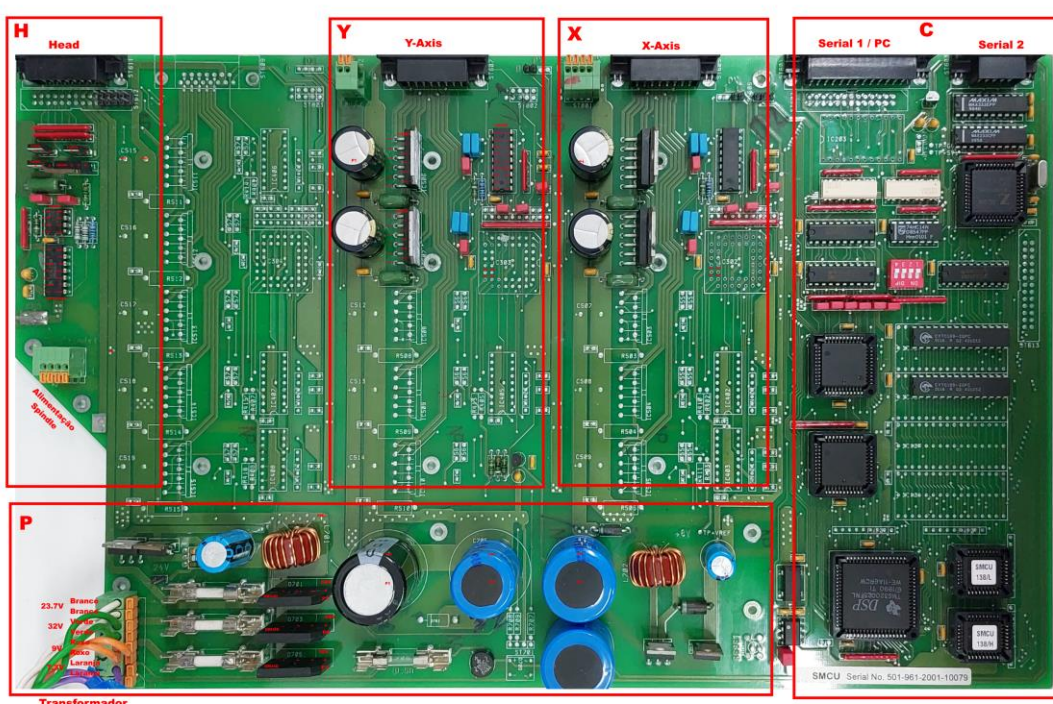


Figura 10- Placa Principal da Fresadora Protomat C60

Estado Inicial: A fresadora apresentava falhas significativas em seu sistema de controle e comunicação com o computador, decorrentes de defeitos em alguns dos circuitos integrados responsáveis por essa função. Além disso, mesmo que o sistema estivesse operacional, a comunicação só poderia ser realizada por meio do software proprietário da fabricante, lançado há mais de vinte anos, o que limita severamente sua utilização e compatibilidade com sistemas atuais.

Conforme indicado na Figura 10, devido à inoperância da seção C, responsável pelo controle geral da máquina, as seções X e Y, encarregadas do acionamento dos motores de passo dos

respectivos eixos, também se tornaram inutilizáveis. O mesmo ocorre com a seção H, responsável pelo controle do solenoide.

Dessa forma, a única parte aproveitável da placa original é a seção P, utilizada para fornecer alimentação à placa de potência dos MOSFETs do spindle, operando com tensões de 45V e 5V.

4.1.2 Placa Spindle

A Figura 11 apresenta a placa do spindle, responsável por gerar as 3 fases senoidais que são usadas para controlar a velocidade de rotação do spindle. Através do chaveamento dos MOSFETs da forma correta se faz o motor girar.

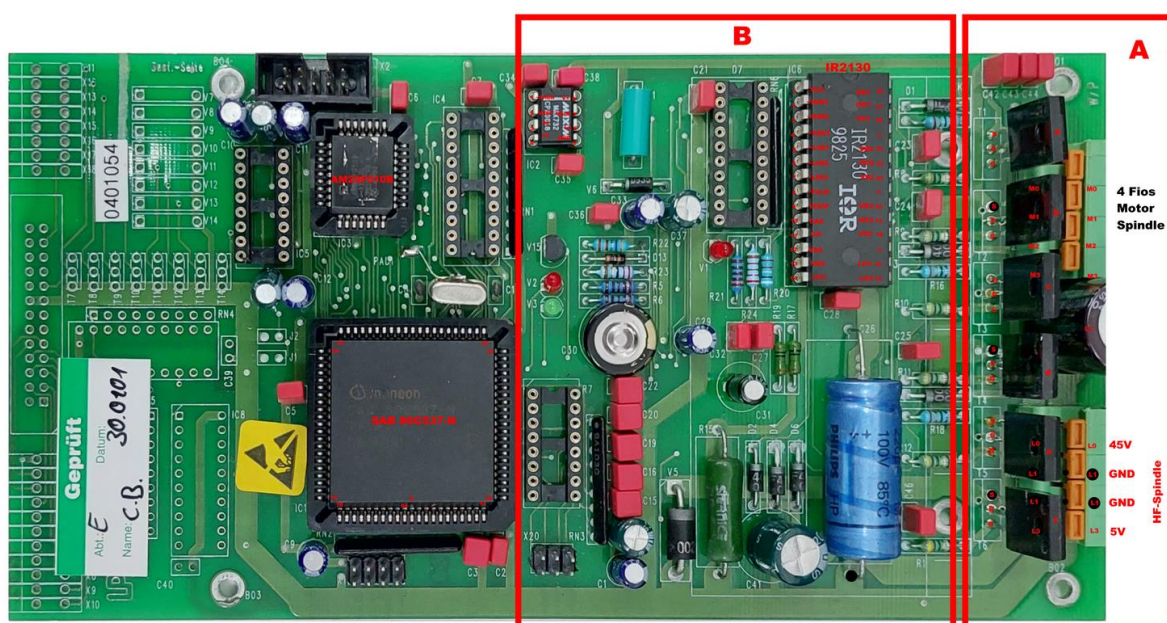


Figura 11- Placa do Spindle da Protomat C60

Estado Inicial: Sem funcionar e faltando componentes. Os MOSFETs e o driver IR2130 ainda funcionam, mas o cérebro desta placa não existe mais. Ela será aproveitada, usaremos um Arduino Uno para gerar as 3 ondas senoidais que controlam o chaveamento dos MOSFETS para controlar o spindle.

4.1.3 Motores de Passo

Esta fresadora tem dois deles, um para movimentar na direção X e outro para direção Y.

Estado Inicial: Os dois estão funcionando corretamente e serão aproveitados.

4.1.4 Solenoide

É usado para mover o cabeçote com a fresa para cima e pra baixo (eixo Z).

Estado Inicial: Funciona corretamente e será aproveitado.

4.1.5 Spindle

É a peça responsável por fazer a Fresagem.

Estado Inicial: Funciona corretamente e será aproveitado.

4.1.6 Interruptores de Fim de Curso

São sensores que detectam quando o eixo da CNC chega ao limite de movimento. Servem para evitar colisões e danos.

Estado Inicial: Funciona corretamente e será aproveitado.

4.2 Escolha do Firmware

A tabela a seguir apresenta uma comparação entre os principais firmwares utilizados no controle de máquinas CNC de pequeno porte. O objetivo é evidenciar suas características, limitações e adequação ao contexto específico do retrofit da fresadora LPKF ProtoMat C60, permitindo justificar de forma técnica a escolha do firmware mais apropriado para este projeto.

Firmware	Plataforma	Rede / Interface	Adequação à LPKF C60
Grbl v1.1	Arduino UNO / Nano (ATmega328p)	USB Serial	Funciona bem, mas exige adaptações externas para controlar o solenoide do eixo Z e o spindle.
FluidNC	ESP32 (ex.: MKS DLC32)	Wi-Fi (Web UI, API REST, Serial)	Excelente opção: integra facilmente o controle do solenoide e do spindle, além de permitir ajuste e monitoramento via Wi-Fi.
Marlin 2.x	MCUs 32 bits (placas de impressoras 3D)	USB Serial / LCD	Pouco prático; complexidade de configuração e respostas mais lentas para comandos de CNC.
Klipper	Raspberry Pi + MCU auxiliar	Rede (WebUI Mainsail/Fluidd)	Inadequado para CNCs autônomas; necessidade de Raspberry Pi torna o sistema mais complexo.

Tabela 1- Comparação entre firmwares

A partir da comparação apresentada, o FluidNC foi escolhido como firmware principal por oferecer uma estrutura moderna, flexível e altamente configurável, características essenciais para um retrofit que exige adaptações específicas. Diferentemente de firmwares mais rígidos, ele permite realizar ajustes diretamente em arquivos de configuração YAML, sem recompilar o código, tornando o processo de calibração mais simples e rápido. Sua arquitetura baseada em

ESP32 também adiciona vantagens importantes, como suporte nativo a Wi-Fi, interface Web integrada e maior desempenho no processamento de comandos G-code, contribuindo para uma operação mais prática e eficiente.

Além disso, por ser um firmware de código aberto, possibilita personalizações profundas e maior liberdade na integração com o hardware existente. Um dos fatores decisivos para sua adoção foi a capacidade de configurar o eixo Z como um solenoide, recurso fundamental neste projeto, já que a fresadora utiliza um sistema eletromecânico de acionamento em vez de um eixo mecânico tradicional. O FluidNC permite tratar esse atuador como uma saída dedicada e totalmente ajustável, garantindo compatibilidade com o funcionamento original da LPKF ProtoMat C60 e simplificando a integração entre o novo controle e a mecânica da máquina.

4.3 Planejamento do Retrofit

O planejamento do retrofit da fresadora LPKF ProtoMat C60 iniciou-se com a definição do sistema de controle principal. Após análise das opções disponíveis, foi escolhido o firmware FluidNC como núcleo do projeto.

Com o firmware definido, foi realizado o planejamento dos componentes intermediários necessários para adaptar os sistemas elétricos e eletrônicos originais da máquina ao novo controlador.

Os motores de passo dos eixos X e Y foram mantidos, sendo controlados por dois drivers TB6600, conhecidos pela robustez e pela capacidade de operar com correntes elevadas e microstepping configurável, garantindo torque adequado e suavidade nos movimentos.

Para o eixo Z, que originalmente utiliza um solenoide em vez de motor de passo, optou-se pelo uso de um módulo MOSFET optoacoplado FR120N. Esse componente atua como interface de potência entre a controladora e o solenoide, permitindo o acionamento seguro do mecanismo de descida da ferramenta, com isolamento elétrico e resposta rápida.

Por fim, o controle do spindle foi planejado de forma independente, utilizando um Arduino Uno em conjunto com a placa original do spindle da ProtoMat. O Arduino foi programado para atuar como uma interface de controle, recebendo comandos de habilitação e velocidade, garantindo que o motor de alta rotação funcionasse conforme as necessidades do processo de usinagem.

Esse conjunto de decisões e adaptações possibilitou a integração de todos os sistemas da fresadora em uma nova arquitetura de controle moderna, acessível e totalmente reconfigurável, mantendo a funcionalidade original da máquina e proporcionando maior flexibilidade para futuras melhorias.

A seguir, é exibida a tabela com os novos componentes adicionados e a sua finalidade.

Componente	Finalidade
MKS DLC32	Nova Placa Principal de Controle CNC
TB6600 (2x)	Driver para controlar o Motor de Passo (2x)
Módulo MOSFET FR120N	Controlar o Solenoide para subir e descer
Arduino Uno	Nova placa de controle do Spindle

Tabela 2- Componentes adicionados

Conexões entre os componentes

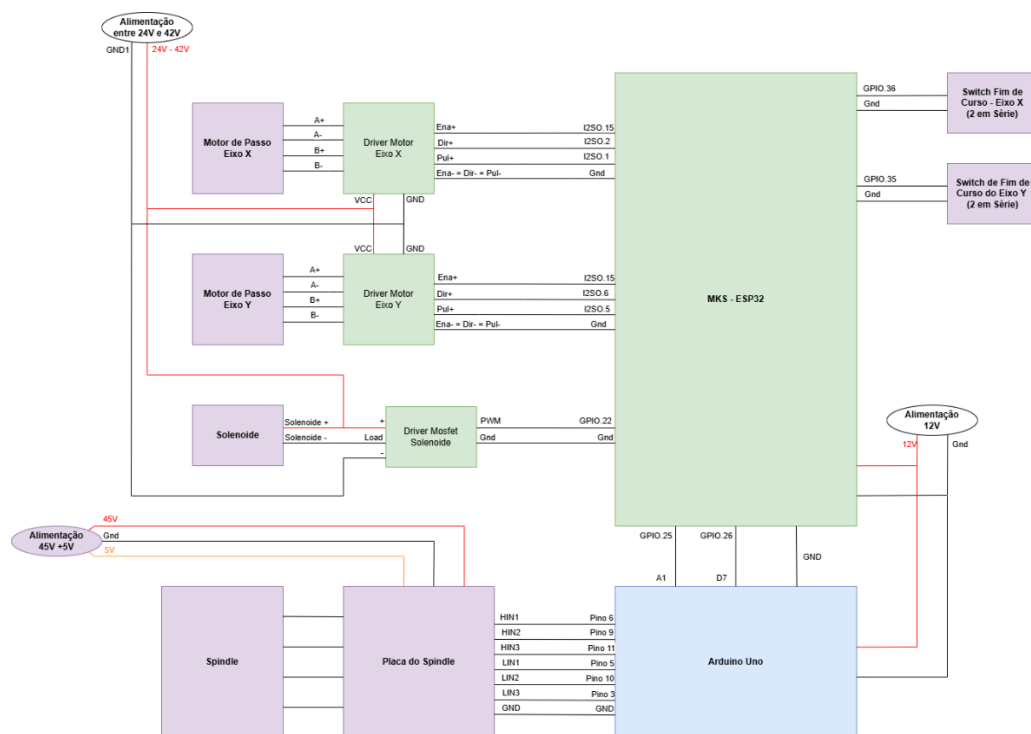


Figura 12- Diagrama de Componente e Conexões

A Figura 12 apresenta o diagrama de conexões planejado entre os principais componentes utilizados no processo de retrofit da fresadora. No esquema, além da representação das ligações, são indicados os nomes dos pinos e sinais responsáveis pela interconexão dos módulos.

Para facilitar a identificação dos elementos, adotou-se a seguinte convenção visual: elipses representam fontes de alimentação, enquanto retângulos correspondem aos outros componentes. Os elementos destacados em roxo correspondem às partes originais da fresadora, já presentes no equipamento. Por sua vez, os componentes indicados em verde e azul representam os módulos adicionados durante o retrofit para restabelecer o funcionamento completo da máquina.

4.4 Montagem e Integração do Sistema

Para as interligações entre os componentes, foram utilizados cabos de rede (par trançado) na maior parte das conexões, por apresentarem boa relação custo-benefício e facilidade de manuseio. Foram empregados conectores RJ45 e adaptadores RJ45 fêmea-fêmea, possibilitando a conexão e desconexão rápida dos módulos. Além disso, foram utilizados cabos de 1,5 mm² para a alimentação elétrica do spindle, garantindo a corrente necessária ao seu funcionamento. Abaixo, é mostrada a imagem no final da montagem.

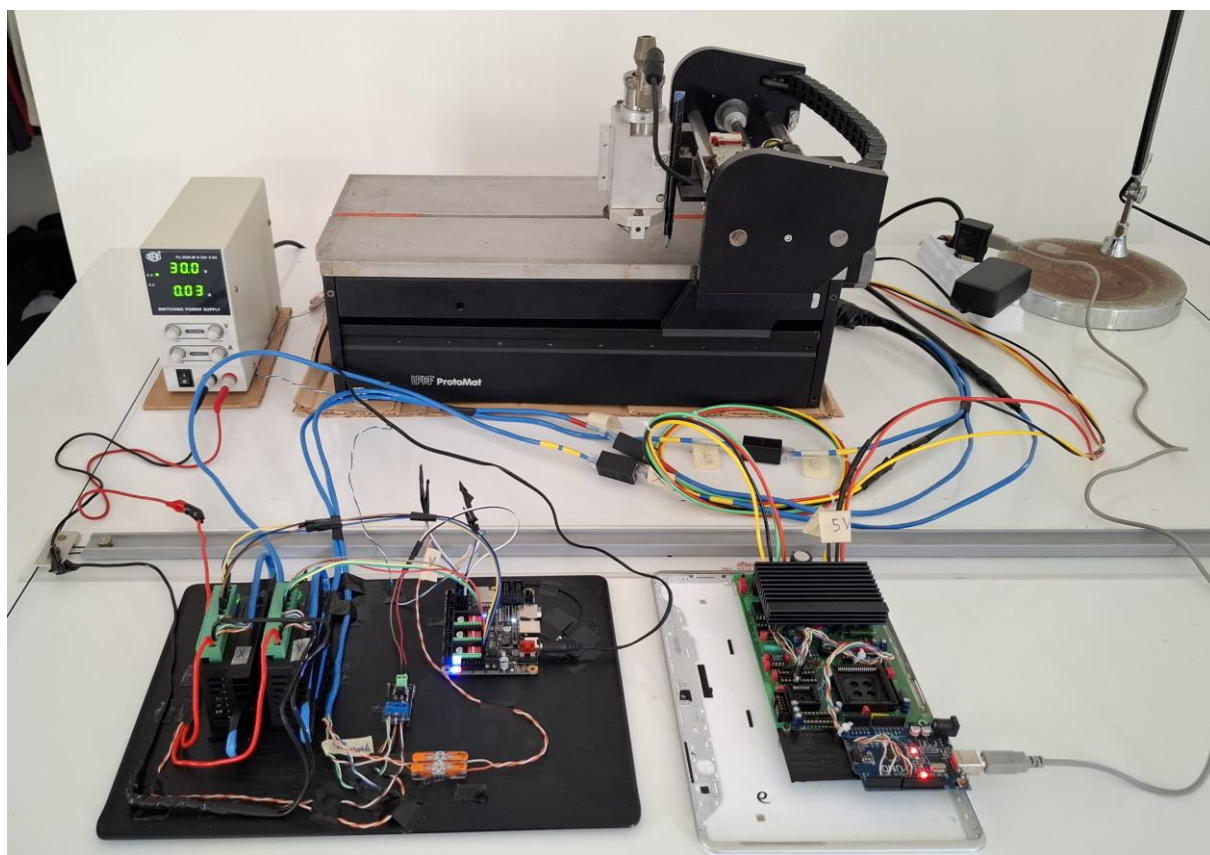


Figura 13 - Fresadora no final da montagem

- Unindo a Placa do Spindle com o seu Novo Controlador

Inicialmente, foram soldados sete fios nos pinos do circuito integrado IR2132, localizado na placa do spindle, correspondentes aos sinais de controle dos MOSFETs, conforme apresentado na Figura 14. Em seguida, foi aplicada pasta térmica e instalado o dissipador de calor sobre os MOSFETs, com o objetivo de garantir uma dissipação térmica adequada, conforme ilustrado na Figura 15.



Figura 14 - Placa do Spindle após soldagem nos pinos necessários



Figura 15 - Placa do Spindle após colocar pasta térmica e dissipador de calor

Posteriormente, os sete fios foram conectados ao Arduino Uno por meio de soldagem, a fim de evitar mau contato nas conexões. Por fim, o Arduino Uno foi fixado à placa do spindle, de modo a evitar esforços mecânicos sobre as soldas, resultando no conjunto em funcionamento, como mostrado na Figura 16.

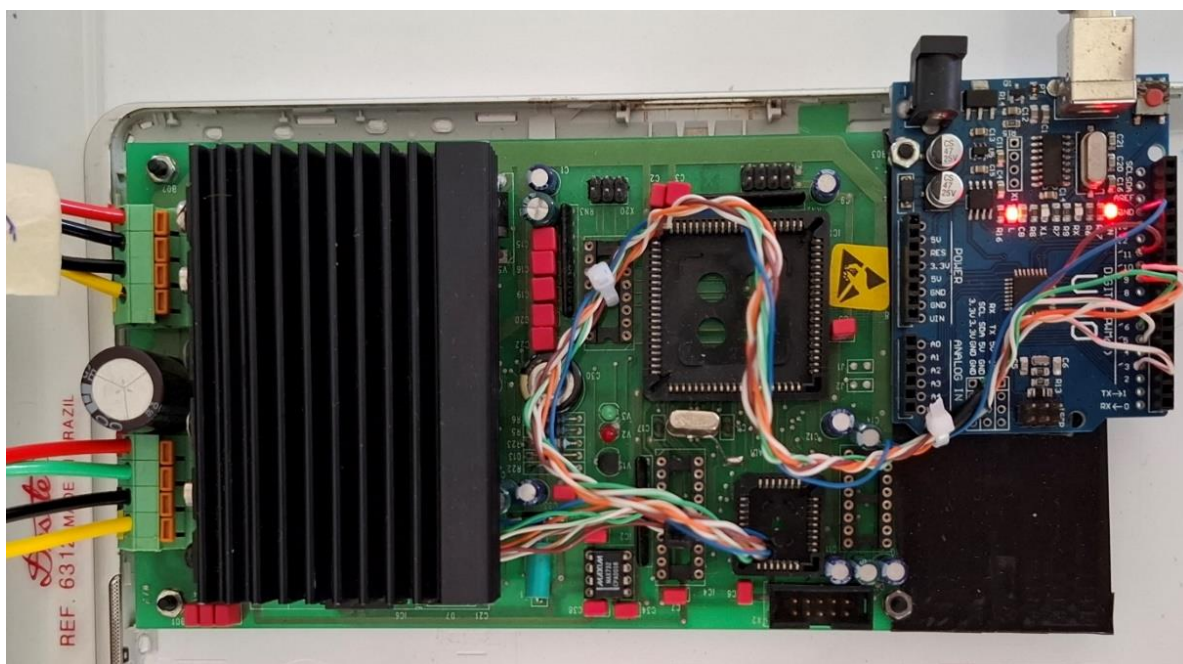


Figura 16 - Placa do Spindle funcionando com Arduino Uno

4.5 FluidNC

4.5.1 Instalação

Para o retrofit da fresadora, utilizou-se o firmware FluidNC versão 3.7.8, instalado diretamente no módulo ESP32 presente na placa MKS DLC32 V2.1. O procedimento executado foi o seguinte:

1. Preparação da placa

A MKS DLC32 necessita de alimentação externa de 12 V para que o ESP32 seja corretamente reconhecido pelo computador via USB.

2. Download do firmware

Foi realizado o download do pacote precompilado do FluidNC 3.7.8 no repositório oficial do projeto. O arquivo ZIP foi integralmente descompactado em uma pasta local.

3. Isolamento da comunicação serial

Para evitar conflitos, todos os outros dispositivos USB foram desconectados, mantendo apenas a MKS DLC32 conectada ao computador.

4. Limpeza da memória

Executou-se o script erase.bat para apagar completamente o ESP32.

5. Instalação do FluidNC

Em seguida, foi executado o arquivo install-wifi.bat, responsável por gravar a versão WiFi do firmware FluidNC. Ao final da instalação, o utilitário FluidTerm é iniciado automaticamente.

6. Envio da interface WebUI

Com o FluidTerm aberto, utilizou-se o atalho CTRL + U para enviar o arquivo index.html.gz, atualizando a interface WebUI do FluidNC.

7. Envio do arquivo de configuração

Novamente com CTRL + U, foi enviado o arquivo config.yaml contendo as configurações específicas da fresadora.

8. Reinicialização da placa

Após o envio dos arquivos, a MKS DLC32 foi reiniciada, permitindo que o ESP32 carregasse o firmware atualizado juntamente com a interface WebUI e o arquivo de configuração.

Após a reinicialização o FluidNC irá criar a rede Wi-Fi “FluidNC” com a senha 12345678.

4.5.2 Configuração

O arquivo config.yaml é o arquivo central de configuração utilizado pelo firmware FluidNC para definir todos os parâmetros operacionais da máquina CNC. Nele são especificados, de forma estruturada e legível, aspectos como o tipo de cinemática empregada, características dos motores de passo, pinos de controle, limites de movimento, acelerações, velocidades, configurações do spindle, além de módulos adicionais como entradas, saídas e sensores. Por ser um arquivo em formato YAML, sua organização baseia-se em indentação e pares chave-valor, o que facilita a leitura e a modificação manual. Todas as definições presentes nesse arquivo são carregadas pelo firmware no momento da inicialização, determinando o comportamento físico e lógico da máquina.

No Apêndice A é apresentado o código completo do arquivo de configuração. Entretanto, ao longo do texto são destacadas apenas as seções mais relevantes para o entendimento do funcionamento do sistema. Cabe ressaltar que, conforme orientado na documentação oficial do FluidNC, comentários não devem ser inseridos na mesma linha das instruções, devendo sempre ocupar uma linha própria para garantir a correta interpretação do arquivo YAML.

- Spindle

```
pwm:
  pwm_hz: 5000
  direction_pin: NO_PIN
  output_pin: gpio.25
  enable_pin: gpio.26
  disable_with_s0: false
  s0_with_disable: true
  spinup_ms: 12000
  spindown_ms: 5000
  tool_num: 0
  speed_map: 0=0% 9999=17% 10000=0% 60000=100%
  off_on_alarm: false
```

O spindle da fresadora foi configurado no arquivo config.yaml do FluidNC para operar por controle PWM utilizando o pino gpio.25, responsável por ajustar a potência aplicada ao motor conforme o valor definido pelo comando S no G-code. Além disso, foi utilizado um pino de habilitação dedicado (gpio.26), permitindo que o spindle seja ativado ou desativado eletronicamente pelo firmware, aumentando a segurança e o controle durante o processo de usinagem. Como o spindle BLDC não necessita de inversão de rotação, o parâmetro `direction_pin` permanece configurado como `NO_PIN`.

Os tempos de aceleração e desaceleração foram ajustados para 12.000 ms (`spinup_ms`) e 5.000 ms (`spindown_ms`), respectivamente. Esses valores garantem que o spindle tenha tempo suficiente para atingir velocidade estável antes do início do corte, além de permitir um desligamento gradual, evitando quedas bruscas de carga e reduzindo o estresse mecânico no conjunto.

O parâmetro `speed_map` define a relação entre a velocidade solicitada (comandos S) e a porcentagem do sinal PWM enviado ao motor. Valores abaixo de 10.000 rpm são tratados como 0% de potência, garantindo que o spindle permaneça desligado fora da faixa segura de operação. A tabela inclui ainda um ponto intermediário em 9999 rpm → 17%, utilizado para ajustar o

comportamento do driver, seguido de uma relação linear entre 10.000 rpm (0%) e 60.000 rpm (100%), permitindo controle contínuo e proporcional da velocidade dentro da faixa nominal do spindle.

As opções `disable_with_s0` e `s0_with_disable` foram configuradas para garantir que o comando S0 não apenas reduza o PWM para zero, mas também desative eletricamente o spindle via pino de enable, fornecendo um desligamento completo e seguro. O parâmetro `off_on_alarm` permanece desativado para evitar desligamentos automáticos durante alarmes não críticos.

Essa configuração proporciona controle refinado do spindle, permitindo ajuste preciso de velocidade, partida suave e comportamento consistente durante a fresagem, atendendo às necessidades específicas do retrofit da máquina.

- Eixo Z – Solenoide

```
z:
  motor0:
    solenoid:
      output_pin: gpio.22:high
      pwm_hz: 1000
      off_percent: 0
      pull_percent: 100
      hold_percent: 70
      pull_ms: 1000
      direction_invert: false
```

No eixo Z, a fresadora não utiliza um motor de passo tradicional, mas sim um solenoide, responsável por acionar verticalmente a ferramenta conforme a lógica de usinagem da Protomat C60. Por isso, a configuração deste eixo no `config.yaml` segue um formato específico do FluidNC para dispositivos acionados por pulsos, e não para movimentação contínua.

O pino de controle do solenoide foi definido em `output_pin: gpio.22:high`, que determina o sinal enviado pela placa controladora para energizar o atuador. Os parâmetros de PWM (`pwm_hz`, `off_percent`, `pull_percent` e `hold_percent`) foram ajustados durante testes, permitindo controlar a força de acionamento e a força de sustentação após o movimento. O valor de `pull_ms` define o tempo necessário para aplicar a força máxima antes de passar para a fase de manutenção, garantindo que o solenoide baixe completamente o cabeçote sem causar impactos mecânicos excessivos. Por fim, a opção `direction_invert` permanece desativada, uma vez que o solenoide

realiza apenas um movimento linear de descida e retorno, não necessitando inversão de direção como nos eixos motorizados.

- Eixos X e Y – Motores de Passo

<pre> x: steps_per_mm: 126.71 max_rate_mm_per_min: 1500.0 acceleration_mm_per_sec2: 150.0 max_travel_mm: 325.000 soft_limits: false homing: cycle: 1 positive_direction: false mpos_mm: -100.0 feed_mm_per_min: 300.000 seek_mm_per_min: 1500 settle_ms: 500 motor0: limit_all_pin: gpio.36 hard_limits: false pulloff_mm: 2.000 stepstick: step_pin: I2S0.1 direction_pin: I2S0.2 </pre>	<pre> y: steps_per_mm: 127.22 max_rate_mm_per_min: 3000.0 acceleration_mm_per_sec2: 300.0 max_travel_mm: 205.000 soft_limits: false homing: cycle: 1 positive_direction: false mpos_mm: -103.000 feed_mm_per_min: 300.000 seek_mm_per_min: 1500 settle_ms: 500 motor0: limit_all_pin: gpio.35 hard_limits: false pulloff_mm: 3.000 stepstick: step_pin: I2S0.5 direction_pin: I2S0.6:high </pre>
---	--

A configuração dos eixos X e Y no arquivo config.yaml define os parâmetros de movimentação da fresadora retrofitada, garantindo que o FluidNC traduza corretamente os comandos de deslocamento para o movimento físico dos motores de passo. Os valores de steps_per_mm foram obtidos após o processo de calibração da máquina, assegurando precisão dimensional durante a usinagem.

No eixo X, o valor de 126,71 steps/mm foi ajustado com base nas medições experimentais, garantindo que a distância percorrida corresponda exatamente ao deslocamento solicitado no G-code. A velocidade máxima foi limitada a 1500 mm/min, com aceleração de 150 mm/s², valores adequados para evitar perda de passos e garantir operação suave considerando a mecânica original da Protomat C60. O curso total do eixo permanece em 325 mm, e os soft limits foram mantidos desativados para permitir maior flexibilidade durante testes e ajustes.

O procedimento de homing do eixo X foi configurado para atuar no ciclo 1, movendo-se no sentido negativo até acionar o fim de curso conectado ao pino gpio.36. Após o acionamento, o firmware aplica um recuo (pulloff_mm) de 2 mm e estabelece a posição de máquina em -100 mm (mpos_mm), valor que corresponde à referência física utilizada no retrofit. As velocidades

de aproximação (feed) e busca rápida (seek) foram definidas em 300 mm/min e 1500 mm/min, respectivamente, garantindo precisão sem comprometer a rigidez mecânica do conjunto. O motor utiliza os pinos I2SO.1 (step) e I2SO.2 (direction), conforme o mapeamento da MKS DLC32.

O eixo Y segue a mesma estrutura de configuração, porém com ajustes específicos ao seu curso e características mecânicas. O valor de 127,22 steps/mm foi determinado após calibração, e a velocidade máxima foi configurada em 3000 mm/min, o dobro da velocidade do eixo X, uma vez que o deslocamento no eixo Y apresenta menor massa móvel. A aceleração foi mantida em 300 mm/s², proporcionando movimentação suave e estável. O limite físico do eixo permanece em 205 mm, com soft limits desativados.

O homing do eixo Y também ocorre no ciclo 1, movendo-se no sentido negativo até atingir o fim de curso no pino gpio.35. Após o acionamento, o firmware realiza um recuo de 3 mm e define a posição de máquina em -102,5 mm, valor correspondente ao ponto de referência estabelecido durante o ajuste mecânico da mesa da Protomat C60. Os pinos do motor foram configurados como I2SO.5 (step) e I2SO.6:high (direction), sendo necessária a inversão lógica do sinal de direção (:high) para garantir que o eixo Y se mova no sentido correto durante os deslocamentos.

Essa configuração garante que ambos os eixos operem com precisão, repetibilidade e compatibilidade com a mecânica original da fresadora, atendendo às exigências do processo de fabricação de placas de circuito impresso.

- Configuração dos Barramentos I2SO, SPI e Cartão SD

```
i2so:
  bck_pin: gpio.16
  data_pin: gpio.21
  ws_pin: gpio.17

spi:
  miso_pin: gpio.12
  mosi_pin: gpio.13
  sck_pin: gpio.14

sdcard:
  cs_pin: gpio.15
  card_detect_pin: gpio.39
```

A seção I2SO define os pinos necessários para habilitar o modo I2S do ESP32, que é utilizado pelo FluidNC para gerar sinais de passo de alta precisão destinados aos drivers dos motores. Os pinos bck_pin, data_pin e ws_pin foram configurados conforme exigido pelo firmware para permitir o uso correto dos pinos especiais I2SO.X presentes na MKS-DLC32.

A configuração spi estabelece os pinos do barramento SPI (miso_pin, mosi_pin e sck_pin), fundamental para a comunicação com o cartão SD. Já a seção sdcard define o pino de seleção do cartão (cs_pin) e o pino de detecção de presença (card_detect_pin), permitindo que o FluidNC acesse arquivos diretamente do cartão, como G-codes e arquivos de configuração.

4.5.3 Interface WEB

Ao conectar-se à rede Wi-Fi criada pelo módulo ESP32 configurado com o FluidNC, o usuário é automaticamente redirecionado para a interface Web do firmware. Caso o redirecionamento não ocorra, a página pode ser acessada manualmente pelos endereços fluidnc.local ou 192.168.0.1.

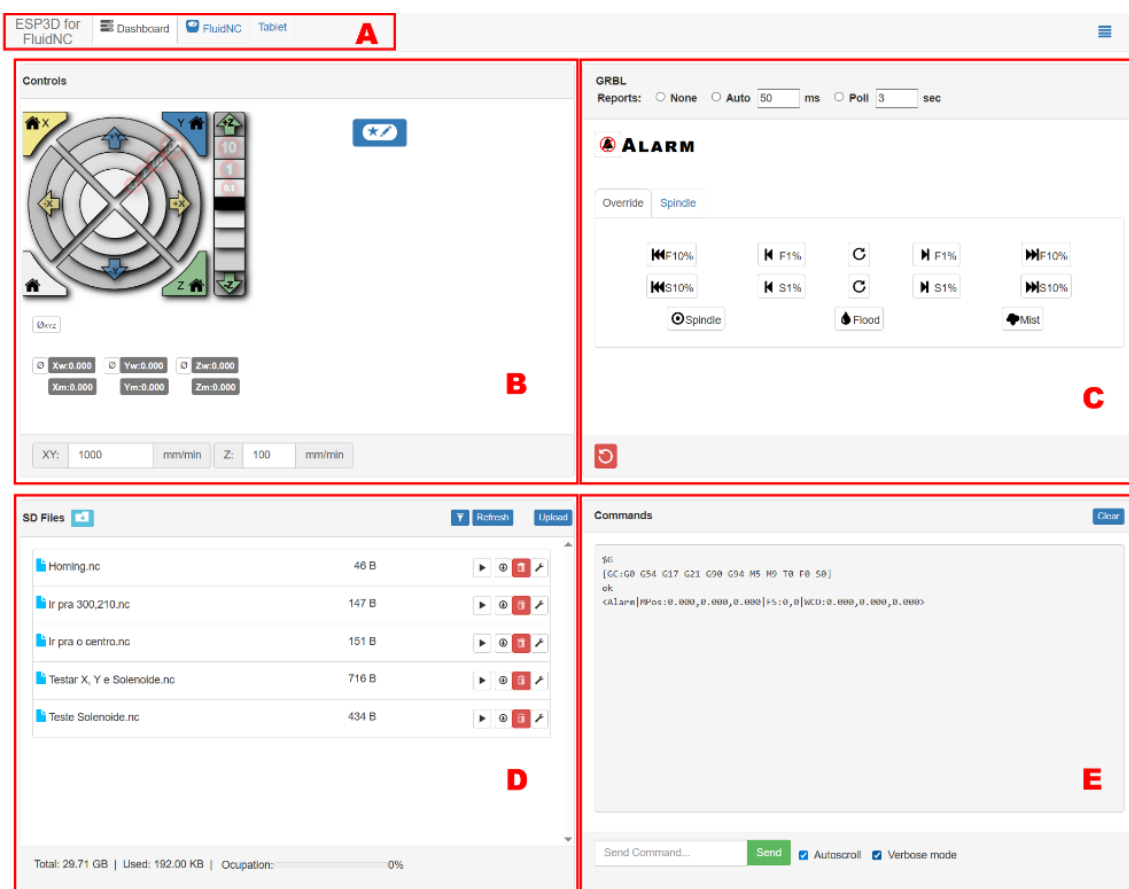


Figura 17- Interface WEB do FluidNC

A Figura 17 apresenta a interface Web e suas principais áreas funcionais, organizadas da seguinte forma:

- Região A – Menu Superior: Fornece acesso às seções de configuração geral do FluidNC, permitindo visualizar e editar o arquivo `config.yaml`, no qual estão definidas todas as parametrizações do firmware.
- Região B – Controle de Posicionamento: Apresenta, em tempo real, as coordenadas atuais dos eixos e disponibiliza controles manuais para movimentação do cabeçote da máquina.
- Região C – Controle de Estado: Responsável pelo gerenciamento do estado operacional da CNC. O FluidNC inicia, por padrão, no modo *Alarm*, impedindo qualquer movimento por segurança. Nessa região, é possível destravar a máquina, alterando o estado para *Idle*.
- Região D – Gerenciamento de Arquivos: Exibe o conteúdo armazenado no cartão SD e permite enviar, organizar e executar arquivos G-code diretamente pela interface.
- Região E – Console de Comunicação: Mostra o log de comunicação entre o firmware e o host, permitindo o envio manual de comandos em G-code e o monitoramento da execução em tempo real.

4.6 Controlador Eletrônico de Velocidade Desenvolvido

Para o controle do spindle da fresadora retrofitada, foi desenvolvido um Controlador Eletrônico de Velocidade (ESC) baseado no código utilizado na monografia do orientador do trabalho, Prof. Juliano Pacheco, o qual foi adaptado para atender às características do motor BLDC presente na LPKF ProtoMat C60. Todo o código-fonte implementado encontra-se disponível no Apêndice B deste documento.

Funcionamento Geral do ESC

O ESC foi implementado em um Arduino Uno R3, empregando acesso direto aos registradores do microcontrolador ATmega328P. Essa abordagem elimina dependência das bibliotecas padrão da plataforma Arduino, reduzindo latências e garantindo maior precisão temporal no acionamento das fases do motor.

O algoritmo utiliza a técnica de DDS (Direct Digital Synthesis) para gerar as três ondas senoidais defasadas necessárias para o acionamento de um motor BLDC trifásico. Uma tabela com 256 valores de seno é armazenada em memória PROGMEM, otimizando o uso de RAM. A cada interrupção do Timer2, o acumulador de fase (phaccu) é atualizado e os valores apropriados da tabela são lidos para compor as três fases com defasagens de 0°, 120° e 240° (offsets de 85 e 170 posições). Esses valores são aplicados aos registradores OCR0A, OCR0B, OCR1A, OCR1B, OCR2A e OCR2B, responsáveis por gerar os sinais PWM entregues ao driver IR2132, que aciona os MOSFETs do circuito de potência do spindle. O uso de interrupções torna o sistema altamente estável, mantendo a atualização contínua do PWM independentemente das demais rotinas executadas no loop principal.

Controle da Velocidade

A rotação do motor é determinada pela variável dfreq, responsável por ajustar o incremento aplicado ao acumulador de fase (tword_m). Valores maiores de dfreq resultam em sinais senoidais mais rápidos e, conseqüentemente, maior velocidade de rotação.

Foi implementada uma rotina de partida suave, que eleva gradualmente a frequência desde valores mínimos até a faixa operacional do spindle (de 10.000 a 60.000 RPM). Essa técnica reduz o estresse mecânico e térmico sobre o motor e aumenta a vida útil dos componentes de potência.

Leitura da Velocidade via ADC e Conversão do PWM da MKS

O comando de velocidade enviado pela placa MKS DLC32 é fornecido na forma de um sinal PWM de 5 kHz. No entanto, o microcontrolador ATmega328P necessita de um valor analógico contínuo para realizar o mapeamento da velocidade desejada.

Para realizar essa conversão, foi implementado um filtro passivo RC, composto por: Um resistor de 10 kΩ e um capacitor de 10 μF.

Esse filtro atua como integrador, convertendo o PWM em uma tensão proporcional ao seu duty cycle. O valor analógico resultante é então lido pelo ADC interno do Arduino. Como a MKS opera em nível lógico de 3,3 V, a leitura máxima esperada no ADC é cerca de 675 (em uma escala de 0 a 1023).

Esse valor é convertido para a faixa de frequências permitida pelo spindle por meio da função `mapFast()`. O filtro RC mostrou-se essencial, reduzindo o ruído e proporcionando leituras mais estáveis, o que melhora a precisão do controle de velocidade.

Comunicação Serial

Foi implementado um módulo de UART a 115200 bps diretamente via registradores, utilizado exclusivamente para transmitir mensagens de depuração, incluindo leituras do ADC e a frequência resultante. Embora não interfira no acionamento do motor, essa funcionalidade facilitou significativamente o processo de validação do ESC.

5 RESULTADOS E DISCUSSÕES

5.1 Homing

Para validar o funcionamento do sistema de homing após a configuração do firmware FluidNC, foi elaborado um arquivo com as seguintes instruções, utilizado para acionar o ciclo de homing e, em seguida, deslocar a máquina até a posição de referência (home).

\$H	; Fazer Homing
G90 G1 X0 Y0 F1000	; Modo Absoluto e vai pra Home (0,0)
M30	; Finaliza o arquivo

Ao iniciar o processo de homing, a fresadora movimentou-se nos eixos X e Y em direção aos respectivos endstops. Assim que cada eixo tocou o seu interruptor de fim de curso, o movimento foi interrompido conforme previsto. Em seguida, a máquina realizou um recuo de aproximadamente 2 mm, permitindo que os interruptores fossem liberados. Após esse afastamento inicial, a fresadora avançou novamente em direção aos endstops, desta vez em velocidade reduzida, com o objetivo de realizar um toque mais preciso — procedimento padrão conhecido como homing seek seguido de homing feed, garantindo maior exatidão na definição da origem mecânica.

Após o segundo toque, os eixos recuaram novamente, finalizando o ciclo de homing. De acordo com os parâmetros definidos no arquivo config.yaml, a posição atingida após o homing corresponde às coordenadas mecânicas aproximadas de (-100; -102,5), valores estabelecidos para alinhar o zero lógico do sistema com a área útil disponível da máquina.

Com o homing concluído e a origem mecânica reconhecida, o script prosseguiu movendo os eixos X e Y até a posição (0, 0), estabelecida como home de trabalho. Esse deslocamento ocorreu de forma correta e consistente com a configuração aplicada, validando a calibração inicial dos endstops, bem como a interpretação do G-code pelo firmware FluidNC.

Esse teste permitiu confirmar que tanto o ciclo de homing quanto o posicionamento absoluto subsequente estão funcionando adequadamente, garantindo repetibilidade e confiabilidade para as operações de usinagem realizadas na fresadora.

5.2 Testes Funcionais dos Eixos e do Spindle via G-code

- Movimentação dos eixos X e Y via G-code:

Para validar o controle dos eixos X e Y por meio de comandos G-code, foram realizados testes utilizando instruções como G90 para definir o modo de posicionamento absoluto e o comando G1 X225 Y102.5 F1500 para mover a máquina até a posição especificada com velocidade controlada. Também foi verificado o funcionamento do comando G0, utilizado para movimentos rápidos sem preocupação com velocidade de corte, em contraste com o G1, que realiza deslocamentos interpolados com taxa de avanço definida. Durante os ensaios, todos os movimentos executados pela fresadora ocorreram corretamente, alcançando as coordenadas programadas sem desvios perceptíveis, comprovando o funcionamento adequado do sistema de movimentação após o retrofit.

```
G90
G1 X225 Y102.5 F1500 ; Move pra a posição indicada
```

- Movimentação do eixo Z via G-code:

No caso do eixo Z, cujo acionamento ocorre por meio de um solenoide em vez de um motor de passo, foram realizados testes de movimentação utilizando comandos G-code específicos. Como o sistema interpreta qualquer valor de $Z \geq 0$ como posição elevada e $Z < 0$ como posição abaixada, empregou-se o comando G90 para definir o modo absoluto, seguido de G1 Z-1 F1500 para acionar o solenoide e abaixar a ferramenta. Em seguida, utilizou-se G4 P1 para inserir um atraso de 1 segundo, garantindo tempo suficiente para o movimento, e posteriormente o comando G1 Z0 para retornar o solenoide à posição levantada. Nos ensaios realizados, o comportamento do eixo Z ocorreu conforme esperado, com a ferramenta sendo acionada e recolhida corretamente, validando o funcionamento adequado do sistema de elevação e acionamento após o retrofit.

```
G90
G1 Z-1 F1500 ; Baixa o solenoide
G4 P1      ; Aguarda 1s
G1 Z0 F1500 ; Levanta o solenoide
```

- Acionamento do Spindle

Para validar o controle do spindle pelo firmware FluidNC, foram realizados testes utilizando comandos G-code específicos para acionamento e desligamento. O comando M3 S10000 foi empregado para ligar o spindle e ajustá-lo a 10.000 RPM, permitindo verificar a resposta do sistema de controle de velocidade. Em seguida, utilizou-se G4 P10 para inserir uma pausa de

10 segundos, garantindo a observação do comportamento estável do motor durante a rotação. Por fim, o comando M5 foi executado para desligar o spindle. Nos testes realizados, o acionamento, a manutenção da rotação e o desligamento ocorreram corretamente, demonstrando que o controle do spindle após o retrofit está funcionando de forma adequada e confiável.

M3 S10000	; Liga o spindle a 10.000 RPM
G4 P10	; Aguarda 10 segundos (P em segundos no FluidNC)
M5	; Desliga o spindle

5.3 Calibração dos Eixos da Fresadora

Para garantir a precisão dimensional da fresadora após o retrofit, realizou-se um procedimento de calibração dos eixos X e Y com base na medição prática do deslocamento real da máquina. A calibração teve como objetivo ajustar o parâmetro `steps_per_mm` de cada eixo, de forma que o movimento comandado pelo firmware correspondesse exatamente ao movimento físico executado. O método adotado consistiu em fixar uma caneta ao cabeçote da máquina utilizando fita dupla face, transformando temporariamente o equipamento em um plotter.

A calibração foi realizada por meio da execução de um percurso controlado, utilizando dois retângulos como referência métrica, conforme ilustrado na Figura 18. Cada retângulo possuía 10 mm de largura, sendo um com 200 mm de comprimento no eixo X e outro com 100 mm de comprimento no eixo Y, permitindo verificar o deslocamento real da máquina em relação ao deslocamento nominal programado. O arquivo G-code utilizado para essa etapa encontra-se no Apêndice D.

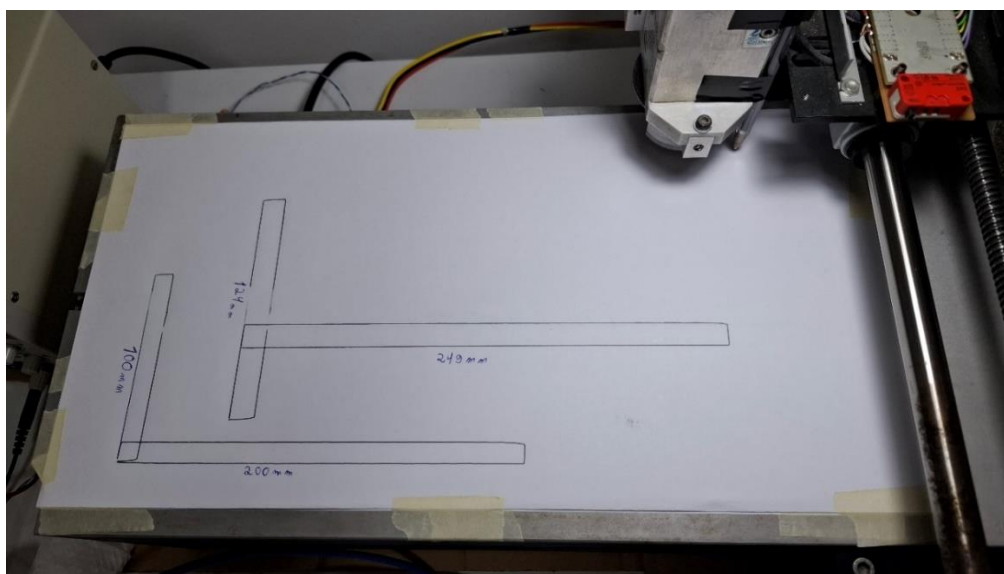


Figura 18 – Retângulos de calibração antes e após calibrar.

Antes da calibração, ambos os eixos utilizavam o parâmetro $\text{steps_per_mm} = 157,75$, valor inicialmente estimado com base nas especificações dos motores e polias. Após a usinagem dos retângulos, foram obtidos os seguintes deslocamentos reais:

- O retângulo de 200 mm no eixo X resultou em um comprimento medido de 249 mm;
- O retângulo de 100 mm no eixo Y resultou em um comprimento medido de 124 mm.

Com esses dados, foi possível ajustar os valores de steps_per_mm aplicando uma proporção direta entre o valor atual e a razão entre deslocamento programado e o obtido. Assim:

$\text{stepsPerMM } X = 157,75 \cdot \frac{200}{249} = 126,71$	$\text{stepsPerMM } Y = 157,75 \cdot \frac{100}{124} = 127,22$
--	--

Após inserir os novos valores no arquivo de configuração e reiniciar a máquina, o teste foi repetido. Os novos retângulos desenhados apresentaram dimensões precisas, confirmando que os eixos passaram a executar deslocamentos fiéis aos valores solicitados, conforme ilustrado na Figura 18, que apresenta os resultados antes e após a calibração. Dessa forma, a calibração dos eixos X e Y foi concluída com sucesso, garantindo maior confiabilidade na execução dos futuros trabalhos de usinagem e isolamento de trilhas.

5.4 Geração dos arquivos G-code a partir do KiCad e FlatCAM

Para validar o fluxo de fabricação na fresadora retrofitada, foram desenvolvidos dois projetos de teste no KiCad 8.0, ambos seguindo os mesmos parâmetros de usinagem definidos para a máquina. O primeiro projeto consiste em uma placa de 20×20 mm, concebida para ser usinada com uma fresa de 0,5 mm. O segundo projeto possui dimensões maiores, 40×40 mm, e foi planejado para uso de uma fresa de 0,9 mm. Em ambos os casos, após o desenho dos layouts, foram exportados os arquivos Gerber (camadas F_Cu e Edge_Cuts) e o arquivo Excellon contendo os furos.

Os arquivos gerados foram importados no FlatCAM 8.5, onde foram convertidos em G-code utilizando os parâmetros $\text{cutZ} = -0,15$ mm e $\text{travelZ} = 2,0$ mm, adequados à máquina que não possui controle ativo de eixo Z. Para o projeto de 20×20 mm foi utilizada uma fresa de 0,5 mm, enquanto o projeto de 40×40 mm utilizou uma fresa de 0,9 mm, ambos com feed rate de 100 mm/min. Os G-codes resultantes foram então enviados diretamente para a fresadora operando com o firmware FluidNC.

As Figuras 19, 20, 21 e 22 apresentam os layouts das placas desenvolvidas no software KiCad:

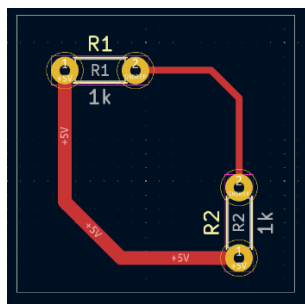


Figura 19– PCB 20mm – Vista da PCB

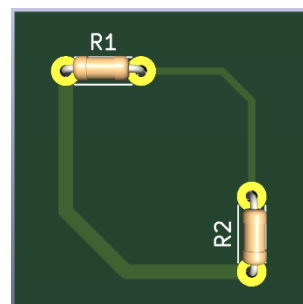


Figura 20 – PCB 20mm - Vista 3D

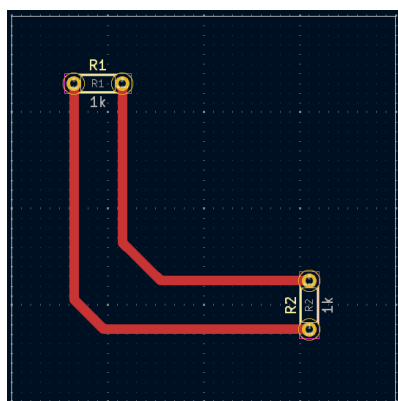


Figura 21- PCB 40mm

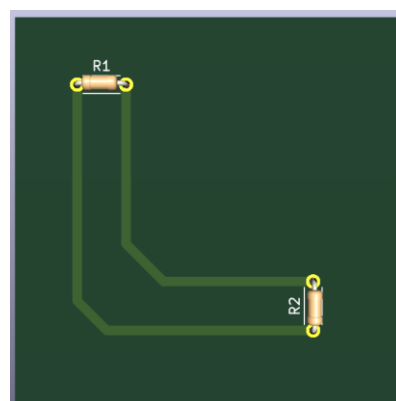


Figura 22- PCB 40mm - Vista 3D

5.5 Validação Preliminar do Processo por Impressão com Caneta

Antes de realizar a fresagem propriamente dita, foi feita uma etapa preliminar de validação utilizando uma caneta acoplada ao cabeçote da máquina. Essa abordagem permitiu testar todo o fluxo de trabalho, incluindo a geração dos G-codes, o envio dos comandos pelo FluidNC e a movimentação dos eixos, sem risco de danificar a placa ou a ferramenta. A impressão com a caneta possibilitou verificar o alinhamento, as dimensões das trilhas e o correto funcionamento dos scripts, garantindo que o processo estava adequado antes de avançar para a usinagem real com a fresa. As Figuras 23 e 24 apresentam os desenhos obtidos nessa etapa de validação.

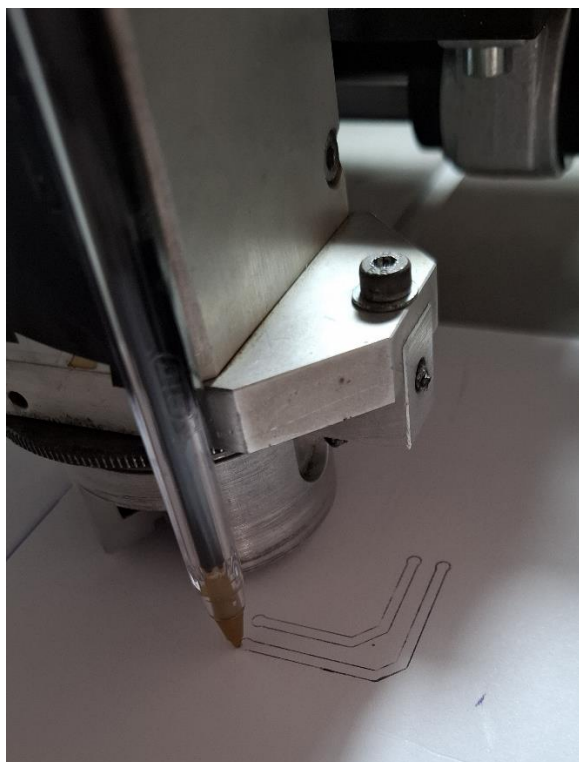


Figura 23- Imprimindo usando caneta

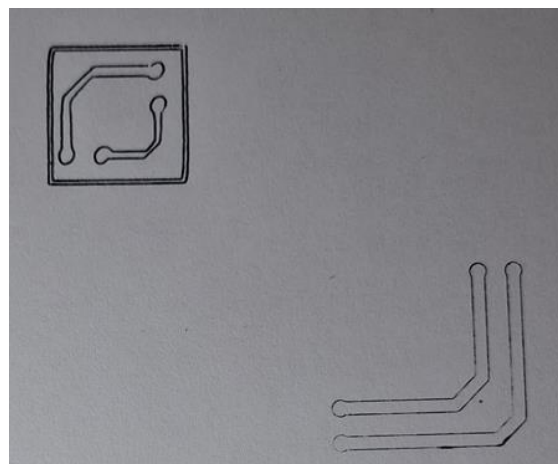


Figura 24- Impressões Realizadas

5.6 Usinagem Final da Placa de Circuito Impresso

Após a validação inicial com a caneta e a confirmação de que o fluxo de geração e execução dos G-codes estava correto, foi realizada a impressão efetiva da placa de circuito impresso utilizando a fresadora retrofitada. Nessa etapa, a máquina executou o percurso de usinagem das trilhas na superfície da placa de cobre, removendo o material de acordo com os contornos definidos nos arquivos Gerber. Em seguida, foram realizados os furos dos componentes conforme o arquivo Excellon, utilizando a broca especificada para cada projeto. Esse processo permitiu verificar a precisão dimensional, o alinhamento das trilhas e a qualidade geral da usinagem. As Figuras 25, 26, 27 e 28 apresentam o resultado final da placa fresada.

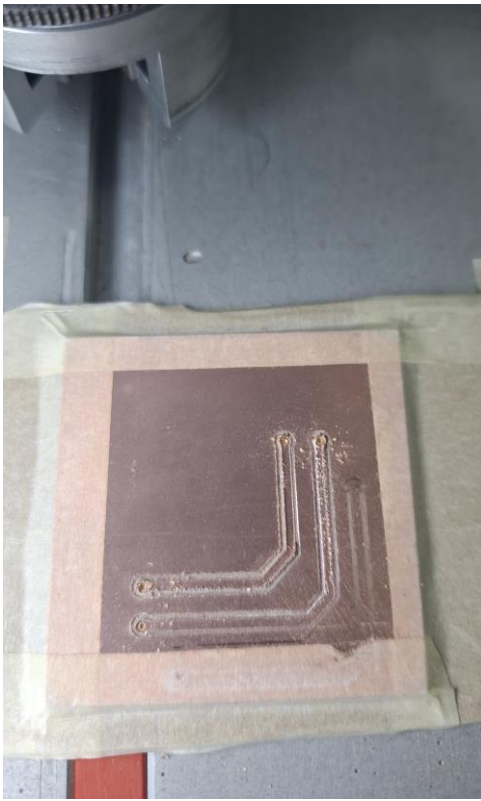


Figura 25- PCB 40mm Finalizada

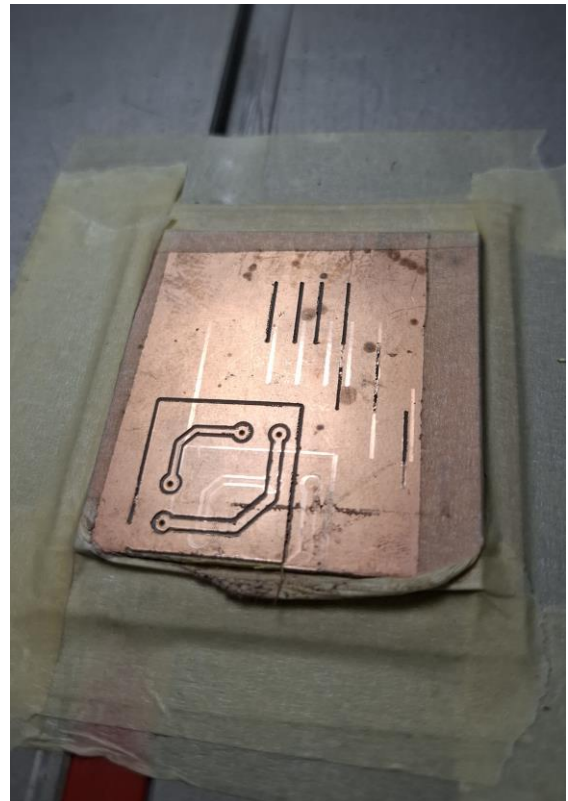


Figura 26- PCB 20mm Finalizada

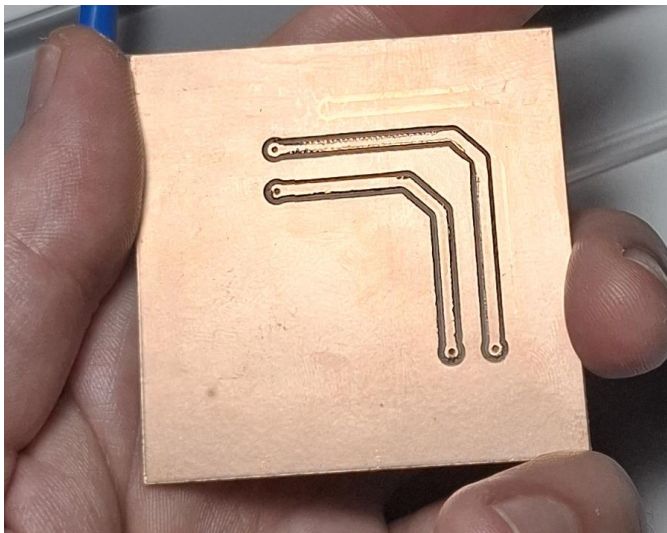


Figura 27- PCB 40mm

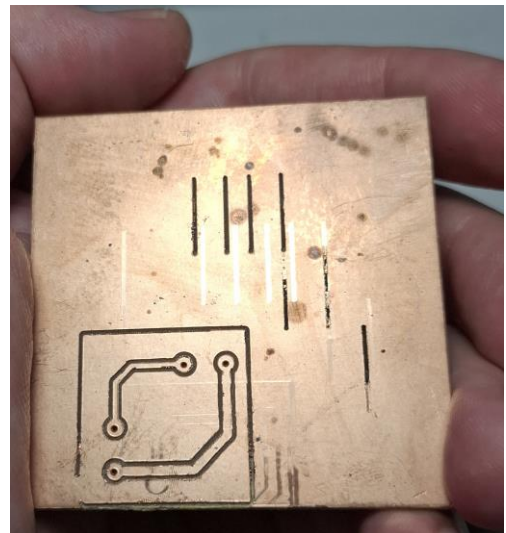


Figura 28- PCB20mm

As Figuras 29 e 30 apresentam a imagem da PCB impressa com a caneta e, ao lado, a mesma placa usinada com a fresa, para fins de comparação.

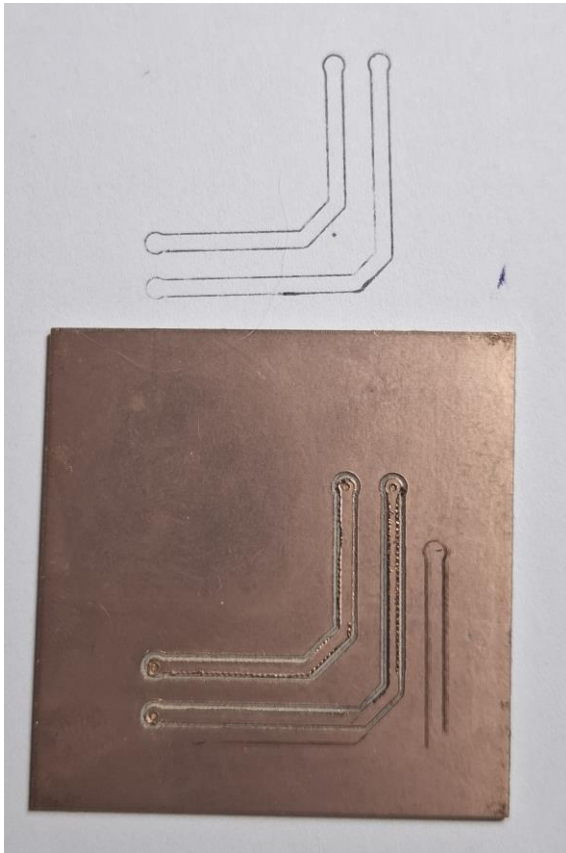


Figura 29- Comparação PCB 40mm

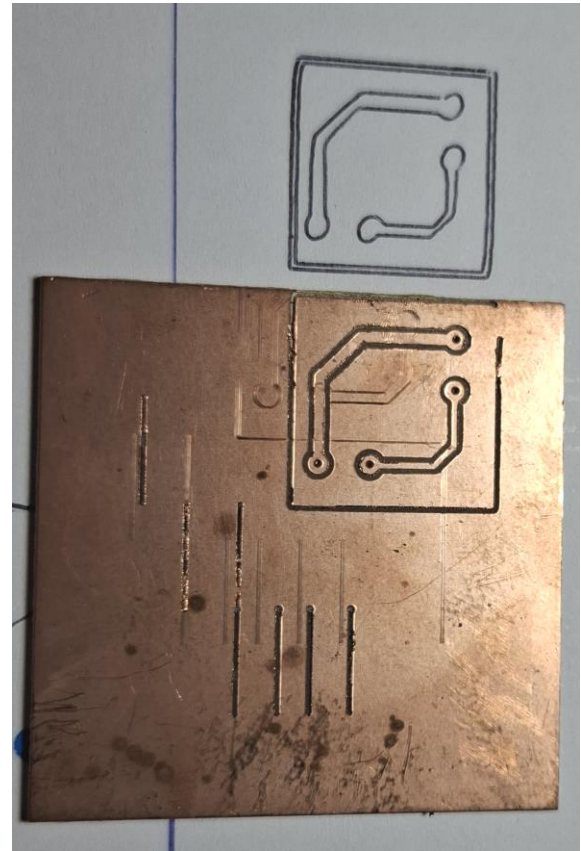


Figura 30- Comparação PCB 20mm

5.7 Propostas de Melhorias para a Fresadora Retrofitada

Durante o desenvolvimento e a validação da fresadora retrofitada, identificaram-se alguns pontos que podem ser aprimorados em trabalhos futuros, visando maior robustez, segurança operacional e desempenho do sistema. Entre as principais melhorias propostas, destacam-se:

- Substituição da fonte de alimentação provisória

Atualmente, o sistema tem operado utilizando uma fonte de bancada, o que, embora funcional, não é adequado para operação contínua ou uso em ambiente produtivo. Recomenda-se a aquisição de uma fonte dedicada, com tensão entre 24 V e 42 V e capacidade mínima de 5 A, garantindo maior estabilidade elétrica, segurança aos componentes e melhor desempenho dos motores de passo.

- Implementação de um sistema de aspiração dedicado

Para controlar a remoção de resíduos gerados durante o processo de fresagem, tem sido utilizado um aspirador de pó doméstico, solução temporária que não oferece eficiência ideal nem segurança para uso prolongado. Assim, uma melhoria importante consiste na instalação de um sistema de aspiração adequado para máquinas CNC, permitindo coleta eficiente de partículas e reduzindo o acúmulo de poeira sobre os componentes mecânicos e eletrônicos.

- Desenvolvimento de um gabinete protetivo para a eletrônica

A operação da fresadora produz grandes quantidades de pó metálico e de fibra de vidro, que podem comprometer a durabilidade e o funcionamento dos módulos eletrônicos. A construção de um case ou gabinete fechado para abrigar o ESP32, drivers, fonte e demais circuitos permitiria proteger o hardware, reduzir riscos de curto-circuito e melhorar a organização interna do sistema.

- Substituição do ESC atual por um controlador com realimentação (BEMF)

O controle do spindle é atualmente realizado por um ESC sem realimentação, o que limita a precisão no controle da rotação, especialmente em variações de carga. Uma melhoria relevante seria a adoção de um ESC com leitura de força contraeletromotriz (Back-EMF), possibilitando controle fechado de velocidade e garantindo maior estabilidade, repetibilidade e qualidade no processo de usinagem.

Essas melhorias, quando implementadas, contribuirão para aumentar a confiabilidade da fresadora, prolongar a vida útil dos componentes e aproximar o equipamento do comportamento esperado de uma máquina CNC profissional.

6 CONCLUSÃO

O projeto apresentado demonstrou que é plenamente viável recuperar uma fresadora de PCBs obsoleta por meio de um processo de retrofit baseado em tecnologias de baixo custo e software de código aberto. A fresadora LPKF ProtoMat C60, antes totalmente inoperante devido à falha irreversível em seu sistema de controle proprietário, pôde ser restaurada funcionalmente através da substituição completa de sua eletrônica de comando e da integração de novos módulos, mantendo-se, entretanto, sua estrutura mecânica original.

A utilização da placa MKS DLC32 com firmware FluidNC mostrou-se uma solução adequada e moderna, oferecendo recursos como interface Web, suporte a G-code, configuração via arquivo YAML e integração simplificada com drivers externos. A adaptação do eixo Z, originalmente acionado por solenoide, confirmou a flexibilidade do FluidNC para lidar com arquiteturas de hardware não convencionais. Além disso, o desenvolvimento de um Controlador Eletrônico de Velocidade (ESC) dedicado, baseado em um Arduino Uno e em técnicas de síntese digital direta (DDS), possibilitou resgatar o funcionamento do spindle BLDC, mesmo sem o controlador original da fabricante.

Os testes experimentais comprovaram o êxito do retrofit. O sistema de homing funcionou corretamente, os eixos X e Y responderam aos comandos G-code com precisão após a calibração, e o spindle apresentou acionamento estável e controle de velocidade funcional. A validação preliminar por impressão com caneta confirmou a coerência geométrica dos trajetos, enquanto a usinagem final das placas demonstrou que a fresadora voltou a ser capaz de produzir PCBs com qualidade adequada para aplicações laboratoriais.

Portanto, o trabalho atingiu plenamente seu objetivo principal: restaurar a fresadora de PCBs e devolvê-la ao uso acadêmico. Além de recuperar um equipamento de alto valor, o projeto agregou novas funcionalidades, maior flexibilidade de uso e facilidade de manutenção. Ademais, sua execução contribuiu para o aprendizado prático em automação, CNC, eletrônica embarcada e integração de sistemas.

Por fim, melhorias como a instalação de um sistema de aspiração, a proteção da eletrônica e o uso de um ESC com realimentação podem elevar a confiabilidade e o desempenho da fresadora. Essas ações prolongam a vida útil do equipamento e aproximam seu funcionamento do padrão de máquinas CNC profissionais. Assim, o retrofit consolida-se como uma solução sustentável e adequada às necessidades de laboratórios de engenharia.

Apêndice A - Arquivo de Configuração do Firmware FluidNC (config.yaml)

```

board: MKS-DLC32 V2.1
name: Retrofit LPKF ProtoMat C60
meta: (26.11.2025) by Gabriel Couto

kinematics:
  Cartesian:

stepping:
  engine: I2S_STATIC
  idle_ms: 0
  pulse_us: 4
  dir_delay_us: 1
  disable_delay_us: 0
axes:
  shared_stepper_disable_pin: I2S0.0
  x:
    steps_per_mm: 126.71
    max_rate_mm_per_min: 1500.0
    acceleration_mm_per_sec2: 150.0
    max_travel_mm: 325.000
    soft_limits: false
    homing:
      cycle: 1
      positive_direction: false
      mpos_mm: -100.000
      feed_mm_per_min: 300.000
      seek_mm_per_min: 1500
      settle_ms: 500
    motor0:
      limit_all_pin: gpio.36
      hard_limits: false
      pulloff_mm: 2.000
      stepstick:
        step_pin: I2S0.1
        direction_pin: I2S0.2
  y:
    steps_per_mm: 127.22
    max_rate_mm_per_min: 3000.0
    acceleration_mm_per_sec2: 300.000
    max_travel_mm: 205.000
    soft_limits: false
    homing:
      cycle: 1
      positive_direction: false
      mpos_mm: -102.500
      feed_mm_per_min: 300.000
      seek_mm_per_min: 1500
      settle_ms: 500
    motor0:
      limit_all_pin: gpio.35
      hard_limits: false
      pulloff_mm: 3.000
      stepstick:
        step_pin: I2S0.5
        direction_pin: I2S0.6:high

```

```
z:
  motor0:
    solenoid:
      output_pin: gpio.22:high
      pwm_hz: 1000
      off_percent: 0
      pull_percent: 100
      hold_percent: 70
      pull_ms: 1000
      direction_invert: true

  pwm:
    pwm_hz: 10000
    direction_pin: NO_PIN
    output_pin: gpio.25
    enable_pin: gpio.26
    disable_with_s0: false
    s0_with_disable: true
    spinup_ms: 12000
    spindown_ms: 5000
    tool_num: 0
    speed_map: 0=0% 9999=0% 10000=17% 60000=100%
    off_on_alarm: false

  i2so:
    bck_pin: gpio.16
    ws_pin: gpio.17
    data_pin: gpio.21

  spi:
    miso_pin: gpio.12
    mosi_pin: gpio.13
    sck_pin: gpio.14

  sdcard:
    cs_pin: gpio.15
    card_detect_pin: gpio.39
```

Apêndice B – Código Arduino

```
#include <avr/io.h> // acesso direto aos registradores (PORT, DDR, TCCR, etc.)
#include <avr/interrupt.h> // controle de interrupções (sei(), cli(), ISR)
#include <avr/pgmspace.h> // para usar PROGMEM e pgm_read_byte_near()
#include <util/delay.h> // para usar _delay_ms() no lugar de delay()

/*#####| DEFINIÇÕES |#####*/

// Frequência de rotação do motor
#define fMax 1000 // 60k RPM (Valor escrito no spindle)
#define fMin 170 // 10k RPM

// Baud rate UART (115200 bps @ 16 MHz → UBRR0 = 8)
#define UBRR0_VALUE 8

// tabela do seno - 256 valores de 0 a 255
const unsigned char seno[] PROGMEM = {127, 130, 133, 136, 140, 143, 146, 149, 152, 155,
158, 161, 164, 167, 170, 173, 176, 179,
182, 185, 187, 190, 193, 195, 198, 201, 203, 206, 208, 211, 213, 215, 217, 220, 222, 224,
226, 228, 230, 232, 233, 235, 237, 238,
240, 241, 242, 244, 245, 246, 247, 248, 249, 250, 251, 252, 252, 253, 253, 254, 254, 254,
254, 255, 255, 254, 254, 254, 254, 253,
253, 252, 252, 251, 250, 250, 249, 248, 247, 246, 244, 243, 242, 240, 239, 237, 236, 234,
232, 231, 229, 227, 225, 223, 221, 219,
216, 214, 212, 209, 207, 204, 202, 199, 197, 194, 191, 189, 186, 183, 180, 177, 175, 172,
169, 166, 163, 160, 157, 154, 150, 147,
144, 141, 138, 135, 132, 129, 125, 122, 119, 116, 113, 110, 107, 104, 100, 97, 94, 91, 88,
85, 82, 79, 77, 74, 71, 68, 65, 63, 60,
57, 55, 52, 50, 47, 45, 42, 40, 38, 35, 33, 31, 29, 27, 25, 23, 22, 20, 18, 17, 15, 14,
12, 11, 10, 8, 7, 6, 5, 4, 4, 3, 2, 2, 1,
1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 16, 17,
19, 21, 22, 24, 26, 28, 30, 32, 34, 37,
39, 41, 43, 46, 48, 51, 53, 56, 59, 61, 64, 67, 69, 72, 75, 78, 81, 84, 87, 90, 93,
96, 99, 102, 105, 108, 111, 114, 118, 121, 124,
127};

double dfreq;
unsigned int i, repets;

// variaveis usadas dentro da interrupção do timer declaradas como volatile
volatile char current_count;
volatile unsigned char duty; // duty - razão cíclica
volatile unsigned long phaccu; // acumulador de fase
volatile unsigned long tword_m; // DDS tuning word m

// 120 graus de defasagem entre as fases
int offset_1 = 85;
int offset_2 = 170;

// Precisa declarar as funções antes de chama-las
void partida();
void adc_init(void);
uint16_t adc_read(void);
long mapFast(long x, long in_min, long in_max, long out_min, long out_max);

// ==== UART ====
void usart_init(void);
void usart_tx_char(char c);
void usart_tx_string(const char *s);
void usart_tx_uint16(uint16_t val);
```

```

/*#####| PROGRAMA |#####*/

int main(void) {
    // Habilita a interrupção pelo timer2 quando houver overflow
    TIMSK2 = (1 << TOIE2); // Seta só o TOIE2 como 1 e o resto como 0
    sei(); // Habilita interrupções globais para as interrupções funcionarem

    // Define os pinos como output
    DDRB = ((1 << PB1) | (1 << PB2) | (1 << PB3)); // Define os pinos 9, 10 e 11 como
output
    DDRD = ((1 << PD3) | (1 << PD5) | (1 << PD6)); // Define os pinos 3, 5 e 6 como output

    // (Opcional) garante PD1 como saída de TX (não é estritamente necessário)
    DDRD |= (1 << PD1); // pino digital 1 (TX) como saída

    /* CONFIGURAÇÃO DOS TIMERS */
    // Timer0 - MODO PWM FASE CORRIGIDA DE 8 BITS
    TCCR0A = ((1<<COM0A1) | (0<<COM0A0) | (1<<COM0B1) | (1<<COM0B0) | (1<<WGM00) |
(1<<WGM02) | (0<<WGM01));
    TCCR0B = (0<<CS02) | (1<<CS01) | (0<<CS00); // liga T/C0 8 prescaler
    // Timer1 - MODO PWM FASE CORRIGIDA DE 8 BITS
    TCCR1A = (1<<COM1A1) | (0<<COM1A0) | (1<<COM1B1) | (1<<COM1B0) | (1<<WGM10);
    TCCR1B = (0<<CS12) | (1<<CS11) | (0<<CS10); // liga T/C1 8 prescaler
    // Timer2 - MODO PWM FASE CORRIGIDA DE 8 BITS
    TCCR2A = (1<<COM2A1) | (0<<COM2A0) | (1<<COM2B1) | (1<<COM2B0) | (1<<WGM20);
    TCCR2B = (0<<CS22) | (1<<CS21) | (0<<CS20); // liga T/C2 com prescaler de 8

    // Inicia o ADC pra ler o pino A1
    adc_init();

    // Inicia UART (NOVO)
    usart_init();
    usart_tx_string("ESC iniciado.\r\n");

    // Dá a partida no motor aos poucos
    partida();

    // Sem o loop o PWM para de atualizar o tword e o motor para
    while (1) {

        // Só muda os dados 1 vez a cada 100 ciclos (senão gasta tempo demais fazendo isso)
        if (repets == 100) {
            uint16_t leitura = adc_read(); // Pega o valor do pino
            // Como o esp32 é 3,3V e o arduino 5V, o valor máximo do ADC será 675
            dfreq = mapFast(leitura, 0, 675, fMin, fMax); // Calcula a freq equivalente
            tword_m = fma(pow(2,32),(dfreq/3921.6),0); // calulate DDS new tuning word
            repets = 0;
            // ===== Envia info pela serial =====
            usart_tx_string("ADC=");
            usart_tx_uint16(leitura);
            usart_tx_string(" F=");
            usart_tx_uint16((uint16_t)dfreq);
            usart_tx_string("\r\n");
        }
        repets++;
    }
}

```

```

/*****| Interrupções |*****/
// Função chamada ao ocorrer interrupção
ISR(TIMER2_OVF_vect) { // interrupção do timer para leitura da tabela de senos
    phaccu=phaccu+tword_m; // Adiciona tuning M word ao acumulador de fase anterior
    current_count=phaccu >> 24; // Pega os 8 bits mais significativos do acumulador de
    fase (phaccu)
    duty=100;
    OCR2A=((pgm_read_byte_near(seno +
(uint8_t)(current_count)))*duty)/100; // Pino 11
    OCR2B=((pgm_read_byte_near(seno +
(uint8_t)(current_count)))*duty)/100; // Pino 3
    OCR1A=((pgm_read_byte_near(seno + (uint8_t)(current_count +
offset_1)))*duty)/100; // Pino 9
    OCR1B=((pgm_read_byte_near(seno + (uint8_t)(current_count +
offset_1)))*duty)/100; // Pino 10
    OCR0A=((pgm_read_byte_near(seno + (uint8_t)(current_count +
offset_2)))*duty)/100; // Pino 6
    OCR0B=((pgm_read_byte_near(seno + (uint8_t)(current_count +
offset_2)))*duty)/100; // Pino 5
}

/*****| FUNÇÕES |*****/
// Usado pra partir o motor
void partida(void) {
    // Inicia com 1hz e vai aumentando por 2s até chegar a 60hz
    dfreq = 1;
    for(i = 1; i < 20; i++) {
        dfreq = 3*i;
        tword_m = fma(pow(2,32),(dfreq/3921.6),0); // calculate DDS new tuning word
        _delay_ms(100);
    }
    // Inicia com 60hz e vai aumentando por 2,2s até chegar a 10 mil RPM
    for(i = 0; i < 11; i++) {
        dfreq = 60 + 10*i;
        tword_m = fma(pow(2,32),(dfreq/3921.6),0); // calculate DDS new tuning word
        _delay_ms(200);
    }
}

/*****| UART |*****/
// Inicializa USART0 para TX em 115200 bps, 8N1, só transmissão
void usart_init(void) {
    // Configura baud rate
    UBRR0H = (uint8_t)(UBRR0_VALUE >> 8);
    UBRR0L = (uint8_t)(UBRR0_VALUE & 0xFF);
    UCSR0A = 0x00; // modo normal, sem multiprocessador
    UCSR0B = (1 << TXEN0); // habilita só transmissor (sem RX, sem interrupções)
    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00); // 8 bits, 1 stop, sem paridade
}
// Envia 1 caractere (bloqueante, mas com interrupções habilitadas)
void usart_tx_char(char c) {
    while (!(UCSR0A & (1 << UDRE0))) {}
    UDR0 = c;
}
// Envia string terminada em '\0'
void usart_tx_string(const char *s) {
    while (*s) {
        usart_tx_char(*s++);
    }
}
}

```

```

// Envia um inteiro 16 bits em decimal (0-65535)
void usart_tx_uint16(uint16_t val) {
    char buf[6]; // até "65535" + '\0' se quisesse, mas aqui mando invertido
    uint8_t i = 0;
    if (val == 0) {
        usart_tx_char('0');
        return;
    }
    while (val > 0 && i < 5) { // Converte para dígitos ao contrário
        buf[i++] = '0' + (val % 10);
        val /= 10;
    }
    while (i > 0) { // Envia na ordem correta
        usart_tx_char(buf[--i]);
    }
}

/*#####| ADC |#####*/

// === Configuração inicial do ADC ===
void adc_init() {
    ADMUX = (1 << REFS0) | (1 << MUX0);
    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
}
// === Leitura rápida do ADC ===
uint16_t adc_read() {
    ADCSRA |= (1 << ADSC); // inicia conversão
    while (ADCSRA & (1 << ADSC)); // espera terminar
    return ADC; // retorna o valor de 10 bits
}
// === Mapeamento rápido (substitui o map() da Arduino) ===
long mapFast(long x, long in_min, long in_max, long out_min, long out_max) {
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

```

Apêndice C – Pinos Placa MKS DLC32

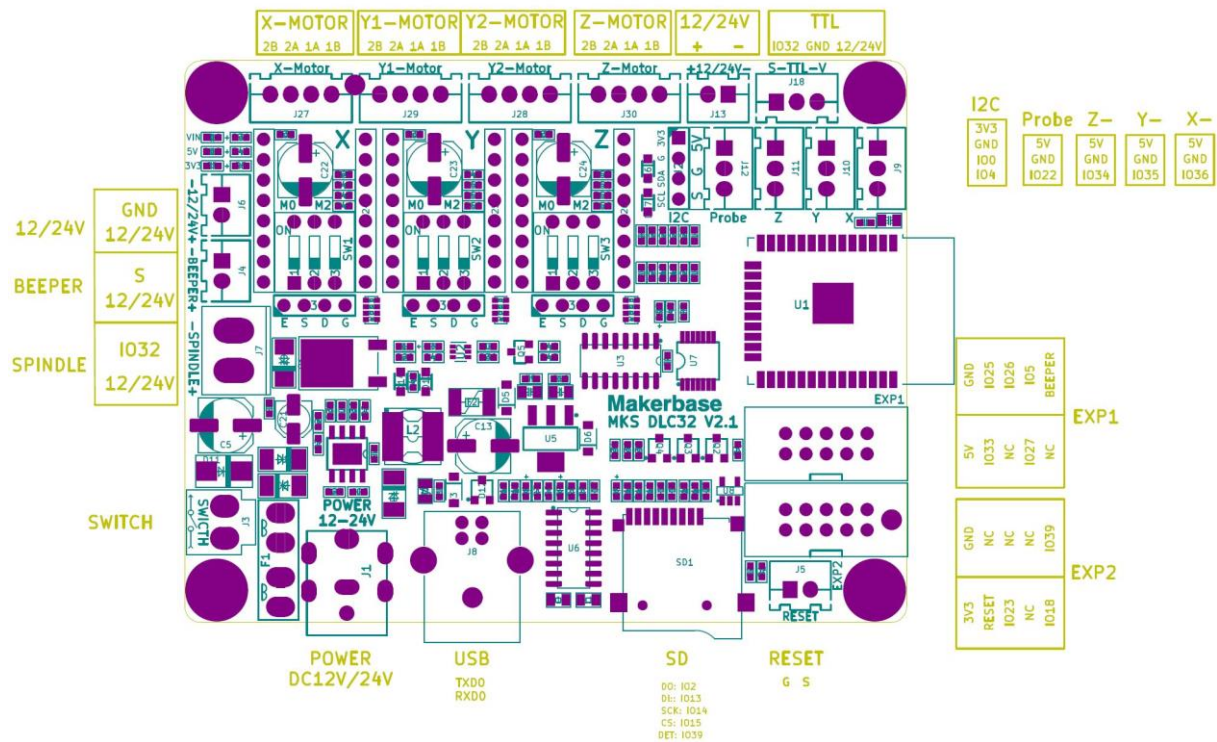


Figura 31 – Pinos da Placa MKS DLC32

GPIO	Disponível	Nome	Pino Externo	Observações
0	✗	I2C_SDA	I2C - SDA	
1	✗	TXD0	USB	Usado para USB/Serial
2		Strapping		
3	✗	RXD0	USB	Usado para USB/Serial
4		I2C_SCL	I2C - SCL	
5	✓	LCD_EN	EXP1	
6 a 11	✗	Flash interno	-	Flash interno - Não Usar
12	✗	miso_pin	Cartão SD	Comunicação SPI (SD Card)
13	✗	mosi_pin	Cartão SD	Comunicação SPI (SD Card)
14	✗	sck_pin	Cartão SD	Comunicação SPI (SD Card)
15	✗	cs_pin	Cartão SD	SDCard
16	✗	bck_pin	-	Comunicação I2SO
17	✗	ws_pin	-	Comunicação I2SO
18	✓	LCD_SCK	EXP2	
19		LCD_MISO	-	

20	✗	-		Não existe
21	✗	data_pin	-	Comunicação I2SO
22	✗	I2C SCL (padrão)	Probe	Controla Solenoide
23	✓	LCD_MOSI	EXP2	
24	✗	-		Não existe
25	✓	LCD_CS	EXP1	
26	✓	TOUCH_CS	EXP1	
27		LCD_RST	-	
28 a 31	✗	-	-	Não existem
32	✓	Spindle or TTL signal	EXP1	
33	✗	LCD_RS	EXP1	Controla Spindle (Conector EXP1)
34	✓	Switches Z	Z-	Só pode ser entrada
35	✗	Switches Y	Y-	Interruptores de fim de curso eixo Y
36	✗	Switches X	X-	Interruptores de fim de curso eixo X
37 e 38	✗	—		Não existem
39	✗	card_detect_pin	EXP2	Pra usar o sdcard

Apêndice D – Script Gcode para calibração (Calibrar.nc):

```

G21          ; mm

;===== EIXO X =====
G90 G1 X-50 Y-20 F1000      ; Ir pra posição de inicio pro eixo X
G1 Z-1 F300                 ; baixa o solenoide
G4 P1                      ; 1s para assentar

; Desenhando Retângulo 250 x 10
G91                      ; coordenadas relativas
G1 X250.0 F600
G1 Y10.0 F600
G1 X-250.0 F600
G1 Y-10.0 F600

G90                      ; coordenadas absolutas
G1 Z1 F300               ; Levanta o solenoide
G4 P0.5                 ; Aguarda 0.5s

;===== EIXO Y =====
G1 X-50.0 Y-60.0 F1000    ; Ir pra posição de inicio pro eixo Y
G1 Z-1.0 F600             ; baixa o solenoide
G4 P1                     ; 1s para assentar

; Desenhando Retângulo 10 x 150
G91                      ; coordenadas relativas
G1 Y150.0 F600
G1 X10.0 F600
G1 Y-150.0 F600
G1 X-10.0 F600

G90                      ; coordenadas absolutas
G1 Z1.0 F600             ; Levanta o solenoide
G4 P0.5                 ; Aguarda 0.5s

M30 ; Finaliza o arquivo

```

Apêndice E – Scripts Gcode Úteis

- Fazer homing e ir pra home (Homing.nc)

```
; ===== FAZER HOMING E DEPOIS VAI PRA HOME (0,0) =====
$H                ; Fazer Homing
G90                ; Modo Absoluto
G1 X0 Y0 F1000     ; Vai pra a Home (0,0)
M30                ; Finaliza o arquivo
```

- Posicionar – Liberar Área da Placa (PosLiberarArea.nc)

```
; ===== LIBERAR AREA DA PLACA - IR PRA LONGE =====
; ORIGEM DA MAQUINA = (-100.0, -102.5)
G21                ; mm
G90                ; coordenadas absolutas
G0 X225 Y102.5     ; vai prra posição indicada
M30                ; Finaliza o arquivo
```

- Posicionar – Trocar Ferramenta (PosTrocarFerramenta.nc)

```
; ===== IR PARA A POSIÇÃO DE TROCAR A FERRAMENTA =====
; ORIGEM DA MAQUINA = (-100.0, -102.5)

G21                ; mm
G90                ; coordenadas absolutas
G0 X-100 Y-102.50  ; Ir pra a posição indicada
M30                ; Finaliza o arquivo
```

- Testar Profundidade da Fresa (TestarProfundidade.nc)

```
; ===== FAZER RASGO PARA TESTAR PROFUNDIDADE =====

; === INICIA SPINDLE E AGUARDA ELE LIGAR ===
G91                ; Coordenadas relativas
M3 S50000          ; Liga o spindle a 50.000 RPM
G4 P10             ; Aguarda 10 segundos (P em segundos no FluidNC)

; === ENTRA NA PROFUNDIDADE DE CORTE ===
G1 Z-1.0 F300      ; baixa o solenoide
G4 P1              ; 1s para assentar

; === FAZ O RASGO ===
G1 X10 F100        ; movimento linear fazendo o rasgo

; === RETRAI E DESLIGA SPINDLE ===
G1 Z1.0 F300
G4 P0.5
M5
M30                ; fim do arquivo
```

REFERÊNCIAS

GAMAZO-REAL, José Carlos; VÁZQUEZ-SÁNCHEZ, Ernesto; GÓMEZ-GIL, Jaime. Position and speed control of brushless DC motors using sensorless techniques and application trends. *Sensors (Basel, Switzerland)*, v. 10, n. 7, p. 6901–6947, 2010. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3231115/>. Acesso em: 26 nov. 2025.

LPKF Laser & Electronics. *C-Series ProtoMat — Technical Data Sheet*. Disponível em: http://www.spezial.cz/pdf/1074-c-series_datasheet.pdf. Acesso em: 26 nov. 2025.

LPKF Laser & Electronics. *ProtoMat C60 — User Manual*. Disponível em: <http://elnmontreuil.free.fr/systemes/LPKF%20install/Bin/Documents/Manuals/ProtoMatC60-e.pdf>. Acesso em: 26 nov. 2025.

Monolithic Power Systems. *Stepper motors — basics, types & uses*. Monolithic Power Systems, [s.d.]. Disponível em: <https://www.monolithicpower.com/en/learning/resources/stepper-motors-basics-types-uses>. Acesso em: 26 nov. 2025.

FluidNC. *FluidNC Documentation (Wiki)*. Disponível em: <http://wiki.fluidnc.com/>. Acesso em: 26 nov. 2025.

PACHECO, Juliano de Oliveira. **Desenvolvimento de um sistema didático para ensino de conversores CC-CA com monitoramento por microcontroladores**. Florianópolis, 2012. Monografia (Especialização em Desenvolvimento de Produtos Eletrônicos) – Instituto Federal de Educação, Ciência e Tecnologia de Santa Catarina. Disponível em: https://professorpetry.com.br/Ensino/Defesas_Pos_Graduacao/Defesa%2020_Juliano%20de%20Oliveira%20Pacheco_Desenvolvimento%20de%20um%20Sistema%20Didatico%20para%20Ensino%20de%20Conversores%20CCCC%20com%20Monitoramento%20por%20Microcontroladores.pdf. Acesso em: 26 nov. 2025.